RSSGL: Statistical Loss Regularized 3-D ConvLSTM for Hyperspectral Image Classification

Liguo Wang, *Member, IEEE*, Heng Wang, Lifeng Wang, Xiaoyi Wang, Yao Shi, and Ying Cui

***Abstract*— Studies on the classification of hyperspectral images (HSIs) based on deep learning are in full swing, especially the spectral–spatial dependent global learning (SSDGL) framework, which is both efficient and robust. However, the global convolutional long short-term memory (GCL) module under this framework fails to take full consideration of the spectral characteristics contained in HSIs, and the hierarchically balanced (H-B) sampling strategy introduced in this framework prevents the training process from converging smoothly. In this article, we develop a novel regularized spectral–spatial global learning (RSSGL) framework. Compared with SSDGL, the proposed framework mainly makes three improvements. Above all, aiming at the problem that the GCL module used in SSDGL cannot fully tap the local spectral dependence, we apply 3-D convolution to the gated units of long short-term memory (LSTM) as an alternative to the GCL module for adjacent and nonadjacent spectral dependencies learning. Furthermore, to extract the most discriminative features, an improved statistical loss regularization term is developed, in which we introduce a simple but effective diversity-promoting condition to make it more reasonable and suitable for deep metric learning in HSI classification. Finally, to effectively address the performance oscillation caused by the H-B sampling strategy, the proposed framework adopts an early stopping strategy to save and restore the optimal model parameters, making it more flexible and stable. Experiments conducted on three representative datasets show that the proposed RSSGL has superior classification performance compared with the existing relatively excellent research methods. The source code is released at https://github.com/swiftest/RSSGL.**

***Index Terms*— 3-D convolutional long short-term memory (ConvLSTM), global learning, hyperspectral image (HSI) classification, spectral–spatial features, statistical loss.**

I. Introduction

HYPERSPECTRAL imaging has always played an extremely important role in remote sensing data analysis. Since this type of image encodes rich spectral and spatial information in many narrow and continuous spectral bands, it can better describe the distribution and types of ground objects and has been applied to many fields, such as agricultural disaster monitoring, mineral detection, change detection, and resource management [1]–[3]. As an extremely important application of hyperspectral image (HSI), HSI classification aims to assign a unique category label to each individual pixel in the image. Contrary to expectations, the supervised classification of HSI faces severe challenges. Among them, the main point is that the number of available labeled samples is usually limited, which is extremely demanding to obtain a classifier with strong generalization ability. Besides that, there are a large number of spectral bands, which brings up the problem of the curse of dimensionality. Although some dimensionality reduction and band selection [4] methods have been used and studied, some information is lost to a certain extent. The problems mentioned earlier as well as the high intraclass differences and interclass similarities that often appear in HSI data may hinder the effectiveness of the classifier [5]–[7].

There are always more solutions than difficulties, and a lot of effort is devoted to HSI classification. The core ideology of the sparse representation-based classification method mainly lies in that although the dimensionality of the natural signals is very high, the signals belonging to the same class are usually located in a low-dimensional subspace. Therefore, for a test pixel, the given training samples dictionary can be used for sparse reconstruction [8]. A support vector machine (SVM) and its various variants have been widely studied in the past 10–20 years as one of the best classification models. The SVM calculates the interval between the data point and the separation plane to find the classification hyperplane that maximizes the interval and has as few interval violations as possible. The morphological method is a joint spectral–spatial classifier that creates a large feature set by hiring a series of morphological operators with different sizes, which can more effectively distinguish different objects from an original image [9], [10].

With the rapid rise of artificial neural networks in the field of computer vision, natural language processing, and other fields, the classification of HSI based on a deep model has made remarkable achievements [11]–[13]. Conventional classifiers, such as logistic regression (LR) and linear SVMs, can be considered as classifiers with only one hidden layer, because they only need to calculate linear regression coefficients or Lagrangian multipliers (and intercepts), while decision trees or kernel SVMs additionally need to select the optimal partition features or calculate the kernel mapping, which can be regarded as two-layer classifiers [14].

Manuscript received March 14, 2022; revised April 13, 2022; accepted May 9, 2022. Date of publication May 11, 2022; date of current version June 6, 2022. This work was supported by the National Natural Science Foundation of China under Grant 62071084 and Grant 62001434. *(Corresponding author: Heng Wang.)*

The authors are with the College of Information and Communications Engineering, Harbin Engineering University, Harbin 150001, China (e-mail: wangliguo@hrbeu.edu.cn; wanghenghrbeu@hrbeu.edu.cn; wanglifeng_2016@163.com; wangxyi11@126.com; shiyao@hrbeu.edu.cn; cuiying@hrbeu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2022.3174305

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

Chen *et al.* [15] first tried to use training sample patches to train convolutional neural networks (CNNs) for deep feature extraction and classification of HSI. Benefiting from the powerful automatic feature extraction capabilities of CNN, a number of deep CNN models for spectral–spatial joint features extraction based on patches have been proposed [16]–[20]. Owing to that the deep CNNs have lots of weight parameters and large model capacity, they often need plenty of training samples to avoid the problem of overfitting. To alleviate the risk of overfitting and consider the advantages of CNNs and Gabor filters at the same time, Chen *et al.* [21] also proposed to combine Gabor filer with CNN. Spatial information including edges and textures can be effectively extracted by Gabor filters, which greatly reduces the burden of feature extraction of the CNNs. Lee and Kwon [22] proposed a model based on a fully convolutional network (FCN), which uses multiscale filter banks to extract joint spectral–spatial features of different scales. Cao *et al.* [23] investigated a unified Bayesian framework to integrate spectral and spatial information, in which CNN is used to model the posterior class distributions to better use the spatial information, and then, the smooth Markov random field (MRF) prior is applied to the estimated labels to further explore the spatial information as MRFs encourage spatially adjacent pixels to have the same land cover label [24]. Obviously, if there are enough training samples, the deep CNN would provide excellent performance. Li *et al.* [25] proposed a novel pixel-pair method, which greatly expands the size of the training samples set, ensuring that the advantages of CNN can be truly utilized. Roy *et al.* [26] proposed A2S2K-ResNet combined with spectral attention to excavate more discriminative spectral–spatial features. In A2S2K-ResNet, the stacked residual blocks can make the model deeper, and the efficient feature recalibration (EFR) mechanism can help the model effectively model the cross-channel dependency of the feature maps. By the way, in the prior work of Roy *et al.* [27], they proposed HybridSN, which uses the sequential arrangement of 3-D and 2-D CNNs to mine more powerful spectral–spatial information representation. A patch-based method assumes that neighboring pixels have high probability to be the same object. However, the pixels contained in a fixed-size patch do not necessarily belong to the same class, especially for the boundary areas across categories [28]. Xi *et al.* [29] proposed a multidirection network embedded with attentional spectral prior to solve this problem. Although this model works well, it also brings high computational complexity. Recently, Chakraborty and Trehan [30] introduced SpectralNET, which performs four-level wavelet decomposition of the input patch, uses high-frequency and low-frequency information in the spatial domain, and has the same effect as the Octave network [31], which extracts low-frequency and high-frequency features at the same time. Graph convolutional networks (GCNs) are able to model long-range spatial relations between samples in HSI [32], [33], which fail to be considered in CNNs. Hong *et al.* [34] introduced a novel supervised version of GCN (miniGCN), and they integrated features extracted from CNNs and miniGCN to achieve better classification results. Efforts have also been made with deep feature fusion networks (DFFNs) [35], which uses three-level residual block to learn deep fusion features, capsule networks [36] for fast and accurate HSI classification, generative adversarial networks (GANs) [37], [38], deep recurrent neural networks (RNNs) [39], [40], and transfer learning as it can transfer knowledge learned from other domains to the HSI classification task [41], [42].

In addition to the methods mentioned earlier, some insightful deep metric loss-based methods have also been proposed, which dedicate to decrease the intraclass distance and increase the interclass distance in the final embedding feature space. Gong *et al.* [43] introduced a multiscale CNN (MS-CNN) accompany with deep metrics that promotes diversity based on the determinantal point process (DPP) to obtain discriminative features for HSI classification. Liu *et al.* [44] used the Euclidean distance function defined on the embedded feature vector of the training sample pairs as the loss function of the Siamese CNN (S-CNN) to extract features that are more discriminative for classification tasks. Gong *et al.* [45] considered the distribution of each subcategory under the data manifold and the correlation between different subcategories and constructed a deep manifold embedding method (DMEM) as a new training loss, adding class-wise information based on geodesic distance during the training process and obtaining identifiable representation for HSI. Guo and Zhu [46] proposed to use the center loss as a regularization term in conjunction with softmax loss to train a CNN model. Center loss can well minimize the intraclass distance of embedded features, but ignores the influence of other class centers and features, which obviously has a serious limitation in maximizing the interclass distance. Huang *et al.* [47] designed an improved triplet (IT) loss where, for each training sample in a mini-batch, IT selects some of the hardest positive pairs and negative pairs to avoid the influence of the outliers. Wang *et al.* [48] innovatively combined the advantages of the center loss and the angular loss [49] (which was used for image retrieval or person re-identification tasks) to develop sphere loss. Sphere loss not only ensures that the centers are not too far away from the intraclass features, but also makes the centers farthest from each other by introducing a uniformity coefficient that forces the centers to be uniformly distributed over the surface of the hypersphere in the spherical feature space. Although these deep metric losses, which are often used as regularization items, can provide a satisfactory performance, they usually consider the Euclidean distance or angular between sample pairs or sample triplets, which makes the training process vulnerable to unbalanced and limited number of training samples. To this end, Gong *et al.* [50] proposed a statistical loss based on multivariate normal distribution, which can alleviate the uncertainty caused by unbalanced and limited number of training samples. But, in this loss function, the magnitude of the diversity-promoting condition designed to enlarge the interclass variance is much larger than the magnitude of the item used to minimize the intraclass variance, despite the set diversity weight. Obviously, this is very unreasonable.

Recently, Zheng *et al.* [51] proposed a fast patch-free global learning framework (FPGA) for HSI classification, which represents the most advanced model. The FPGA uses a brand-new global learning model called FreeNet, which

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

avoids redundant calculations in the overlapping regions between patches adopted in the patch-based classification models. The core design concept of FreeNet is derived from the convolutional encoder–decoder, although it includes a spectral attention module, it does not consider the long short-term cross-channel spectral dependence. A convolutional long short-term memory (ConvLSTM) has drawn widespread advertence and has shown thriving classification results credited to its natural advantages in modeling long short-term spectral dependencies [52]. Hu *et al.* [53], [54] introduced the ConvLSTM 2-D neural network (SSCL2DNN) to capture the long-range dependencies in the spectrum, and to preserve the structural characteristics of the 3-D cube of hyperspectral data as much as possible, they proposed a spectral–spatial ConvLSTM 3-D neural network (SSCL3DNN) to further boost the classification performance. Later, Zhu *et al.* [55] proposed a spectral–spatial dependent global learning (SSDGL) framework for inadequate and unbalanced HSI classification. The main difference between SSDGL and FPGA lies in that SSDGL employs a global convolutional long short-term memory (GCL) to build the long-range spectral dependencies. Nevertheless, GCL omits any consideration of the local short-range spectral dependency. This will ignore some important local spectral information. In addition, the hierarchically balanced (H-B) sampling strategy developed in SSDGL hinders the smooth convergence of the training process, as it cannot guarantee that the model will see sufficiently diverse training samples for several epochs close to convergence. In summary, to solve the problems listed in the following.

1) Even though GCL performs well in the HSI classification task, it fails to account for dependencies between local short-range spectra.
2) Diversity-promoting condition in the statistical loss is not only complex in construction but also numerically large, which results in model performance being sensitive to the diversity weight.
3) For global learning with unbalanced and limited training samples, the H-B sampling strategy developed by SSDGL framework will cause the model to fail to converge smoothly.

In this article, we explore a regularized spectral–spatial global learning (RSSGL) framework for HSI classification. Inspired by the folded spectrum GAN (FS-GAN) [56], which can discover more spectral information between discontinuous bands, the proposed RSSGL employs 3-D ConvLSTM to extract the short-range and long-range cross-channel spectral dependencies and global spatial information. An improved statistical loss was introduced as a regularization term to perform deep metric learning. In addition, the proposed RSSGL framework incorporates early stopping trick to make it more stable and flexible, which is an important decision that not only saves the training time of the model to a certain extent, but also guarantees that the model can always achieve the best performance. The innovations of this article are mainly reflected in the following aspects.

1) A novel regularized 3-D ConvLSTM global learning frame is developed for the first time to learn adjacent (short-range) and nonadjacent (long-range) spectral independencies and global spatial context.
2) An improved statistical loss regularizer is introduced to force the model to extract the most discriminative features.
3) Early stopping strategy is adopted to solve the problem that the global learning framework cannot converge smoothly due to the H-B sampling strategy.
4) The proposed framework improves the feature representation capabilities and achieves the most advanced classification accuracy with fairly limited training samples on three publicly available and widely used datasets [i.e., Indian Pines (IP), University of Pavia (PU), and Salinas (SA)].

The content of the rest of this article is arranged as follows. Section II details the proposed RSSGL framework. Section III reports and analyzes the experimental results. Section IV discusses the sensitivity, effectiveness, and complexity of RSSGL. Finally, Section V gives the conclusions.

II. Proposed Framework

The patches-based CNN models usually only focus on local spatial features and ignore important nonlocal context features [57], [58]. Relatively, the advantage of global learning lies not only in its efficiency and the learning of global spatial context information. RSSGL takes the encoder–decoder backbone used in FPGA and applies the weighted cross entropy loss developed in SSDGL, which can effectively ameliorate the model's recall ratio for categories with fewer training samples. Although the weighted cross entropy may affect the overall classification accuracy, considering that categories with a few training samples are more difficult to distinguish and may contain more information, we still retain it in the proposed framework. Recent works have shown that the spectral attention model [59], [60] can better capture the nonlinear cross-channel interdependence of the transformed feature map, and the spatial attention model [19], [31] can better capture spatial dependencies of different locations or regions in the feature map and better describe the spatial context of HSI. In this case, the residual-based global joint attention mechanisms (GJAMs) introduced in SSDGL will also be absorbed into RSSGL. Table I lists the mathematical symbols used in this article, and the overall architecture of the proposed RSSGL is shown in Fig. 1, which will be depicted in detail in Sections II-A–II-C.

A. 3-D ConvLSTM

One of the most import characteristics of HSI is that it has a continuous spectrum, which brings possibility and rationality to sequence analysis. Let $X_{\text{orig}} \in \mathcal{R}^{B\times H\times W}$ be a full hyperspectral dataset with height H, width W, and B spectral channels. Due to there are three levels of 2-D convolution operations with a stride of 2 (i.e., downsampling), each level reduces the spatial dimension by half, to make neither filling nor discarding any information in each level of downsampling convolution, the spatial size of X_{orig} is increased to an integer multiple of 16 at the beginning; that is, zeros are padded in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

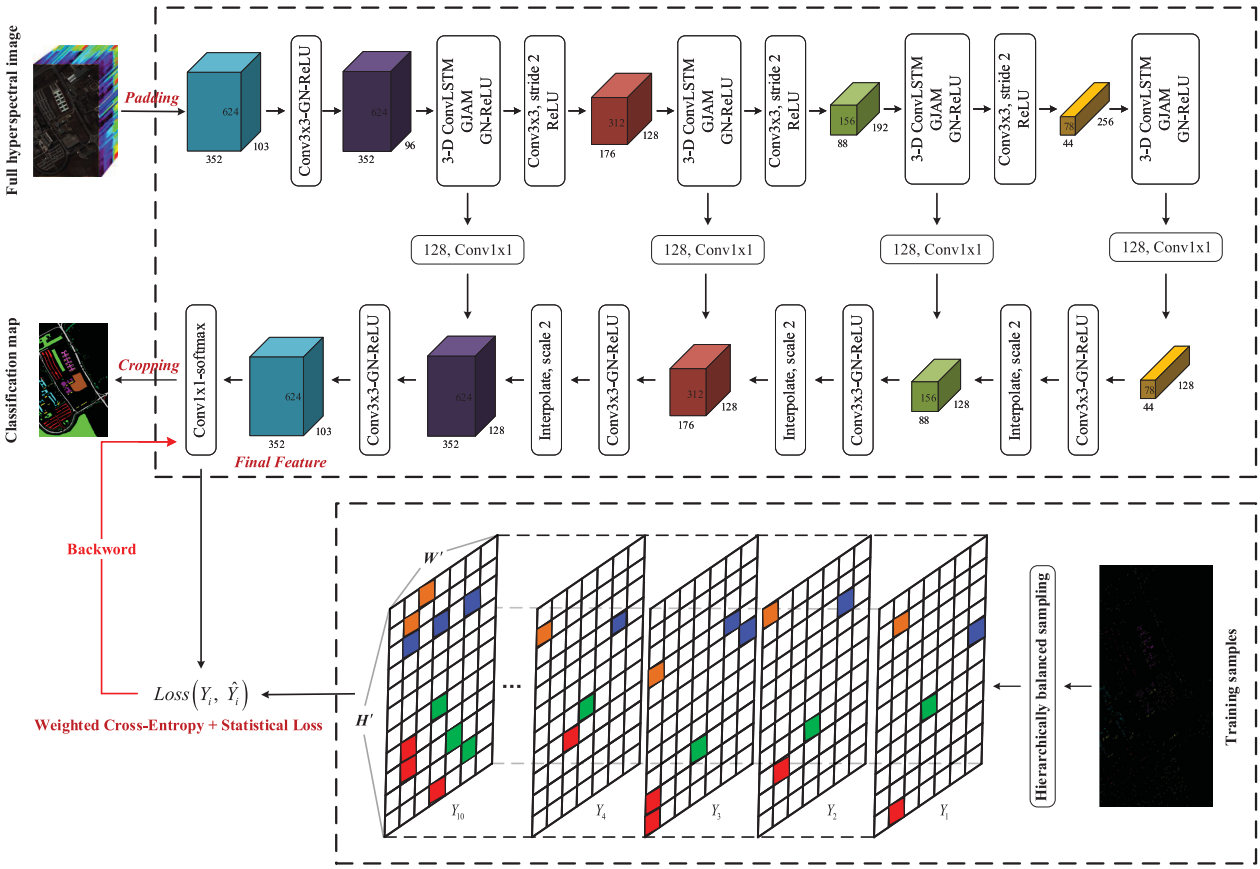


Fig. 1. Overall architecture of the proposed RSSGL. Given a full hyperspectral dataset of size $H \times W \times B$, where B indicates the number of spectral bands, the unified standardized input feature map is passed through 3-D ConvLSTM to learn the short-range and long-range cross-channel dependencies and global spatial context features. Then, abundant spectral–spatial features are extracted through GJAM, and GN is used to correct the inaccurate batch statistics estimation. Finally, the softmax layer is used for classification, and cross entropy combined with statistical loss is used for error backward propagation.

the lower right corner of the space. Here, we mark the padded data cube as $X_{\text{pad}} \in \mathcal{R}^{B\times H'\times W'}$. After that, X_{pad} is fed into a 2-D convolutional layer with 96 convolution kernels of size $B\times 3\times 3$ and a stride of 1. It should be noted that, except for the downsampling operations, we use the "same" padding during both 2-D and 3-D convolutional operations. The obtained unified standardized input feature map with 96 channels can be expressed as

$$F_u = \text{ReLU}((\text{GN}(X_{\text{pad}} * W_{(B\times 3\times 3)} + b))) \tag{1}$$

where ReLU represents the rectified linear unit activation function, GN represents the group normalization (that is, grouping and normalizing on feature channels [51]), $*$ denotes the convolution operation, and $W_{(3\times 3)}$ and b are the weights for the kernel and biases of the convolutional layer, respectively.

Next, the global 3-D ConvLSTM is employed to extract adjacent and nonadjacent spectral dependencies and spatial context features. We divide F_u into eight time steps on the feature channels, i.e., $F_{u1}, F_{u2}, F_{u3}, \ldots, F_{u8}$. Then, $F_{ut} \in \mathcal{R}^{1\times 12\times H'\times W'}$ (here, 1 denotes the batch size) is unsqueezed at the first dimension (e.g., $F_{ut} \in \mathcal{R}^{1\times 1\times 12\times H'\times W'}$) and fed into the 3-D ConvLSTM module, which is shown in Fig. 2. There are two hidden layers in the 3-D ConvLSTM, and it is composed of 3-D ConvLSTMCells, as shown in Fig. 3. In particular, the calculation formulas of the cell can be written as

$$\begin{aligned}
i_{(t)} &= \sigma(W_{xi} * x_{(t)} + W_{hi} * h_{(t-1)} + W_{ci} \otimes c_{(t-1)}) \\
f_{(t)} &= \sigma(W_{xf} * x_{(t)} + W_{hf} * h_{(t-1)} + W_{cf} \otimes c_{(t-1)}) \\
g_{(t)} &= \tanh(W_{xg} * x_{(t)} + W_{hg} * h_{(t-1)}) \\
c_{(t)} &= f_{(t)} \otimes c_{(t-1)} + i_{(t)} \otimes g_{(t)} \\
o_{(t)} &= \sigma(W_{xo} * x_{(t)} + W_{ho} * h_{(t-1)} + W_{co} \otimes c_{(t)}) \\
y_{(t)} &= h_{(t)} = o_{(t)} \otimes \tanh(c_{(t)})
\end{aligned} \tag{2}$$

where t denotes the tth time step, and $x_{(t)}$, $h_{(t-1)}$, and $c_{(t-1)}$ represent the input of the current time step, the short-term state of the output of the previous time step, and the long-term state of the output of the previous time step, respectively. $i_{(t)}$, $f_{(t)}$, and $o_{(t)}$ signify the input, forget, and output gate controllers of the ConvLSTMCell. W_{ci}, W_{cf}, and W_{co} are the peephole weights, W_{xi}, W_{xf}, W_{xo}, and W_{xg} are the gated unit convolution kernels for $x_{(t)}$, and W_{hi}, W_{hf}, W_{ho}, and W_{hg} are the gated unit convolutional kernels for $h_{(t-1)}$. $\otimes$ denotes element-wise multiplication, and the symbol σ represents the sigmoid activation function.

In the initialization phase, the short-term state $h_{(t-1)}$, the long-term state $c_{(t-1)}$, and the peephole weights W_{ci}, W_{cf}, and W_{co} are initialized to zero tensors. In each cell, there are four 3-D convolution kernels in each gated unit for $x_{(t)}$ and $h_{(t-1)}$. When the unsqueezed F_{ut} comes, the output of the first cell

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF MATHEMATICAL NOTATION USED IN THIS ARTICLE

Symbols	Description
$X_{orig} \in \mathcal{R}^{B\times H\times W}$	Original hyperspectral dataset
$X_{pad} \in \mathcal{R}^{B\times H'\times W'}$	Padding X_{orig} with zero
$F_u \in \mathcal{R}^{1\times 96\times H'\times W'}$	Unified standardized input feature map
$F_{u1}, F_{u2}, F_{u3}, \ldots, F_{u8}$	Dividing F_u into 8 time steps
W	Weights for the convolution kernel
$Cell_1$, $Cell_2$	3-D ConvLSTMCells
i_t, f_t, o_t	Input gate, forget gate, and output gate
W_{ci}, W_{cf}, W_{co}	Peephole weights
$W_{xi}, W_{xf}, W_{xo}, W_{xg}$	Gated unit kernels for the input
$W_{hi}, W_{hf}, W_{ho}, W_{hg}$	Gated unit kernels for the state
$\sigma(\cdot)$	Sigmoid activation function
$tanh$	Hyperbolic tangent activation function
t	The time step
$h_{(t)}$, $c_{(t)}$	Short-term state and long-term state
$Y \in \mathcal{R}^{H\times W}$	Ground truth
$Y_{pad} \in \mathcal{R}^{H'\times W'}$	Padded ground truth
Λ	The number of classes in HSI
$N_p(\mu_k, \Sigma_k)$	Multi-variant normal distribution
k, m	Different classes from the iamge
θ	Trainable parameters in RSSGL
B	Training batch
$X_k^B = \{\mathbf{x}_{s_1}, \mathbf{x}_{s_2}, \cdots, \mathbf{x}_{s_{n_k}}\}$	k-th class training samples in B
$C_k^B = \{\mathbf{z}_{s_1}, \mathbf{z}_{s_2}, \cdots, \mathbf{z}_{s_{n_k}}\}$	Extracted features of X_k^B
$\bar{C}_k$	Unbiased estimate of μ_k
$\hat{\Sigma}_k$	Unbiased estimate of Σ_k
L_s, L_{wc}	Statistical loss and weighted cross-entropy
λ	Diversity-promoting weight in L_s
p_j	Weighting factor in L_{wc}
α	Statistical loss weight
N_j	Number of the training samples for class j
$Y_i \in \mathcal{R}^{H'\times W'}$	Mini-batch training samples' labels
$\hat{Y}_i \in \mathcal{R}^{\Lambda\times H'\times W'}$	The output of the softmax layer
$M_1, M_2, \ldots, M_{10}$	Mini-batch training samples' mask
c, P	Early stopping counter and "patience"
η_0	Base learning rate
E	Training epochs

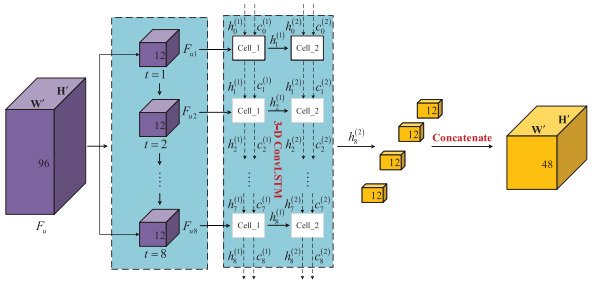


Fig. 2. Architecture of the 3-D ConvLSTM.

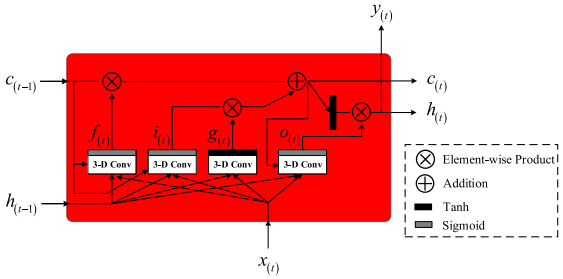


Fig. 3. Architecture of the 3-D ConvLSTMCell.

can be expressed as

$$h_t^{(1)} = \text{Cell_1}(F_{ut}, h_{(t-1)}, c_{(t-1)}). \tag{3}$$

Then, $h_t^{(1)} \in \mathcal{R}^{1\times 4\times 12\times H'\times W'}$ will become the input of the second cell at each corresponding time step. We only select the output of the last time step of the second cell (Cell_2), i.e.,

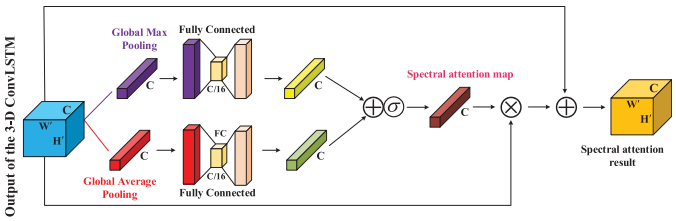


Fig. 4. Global spectral attention mechanism.

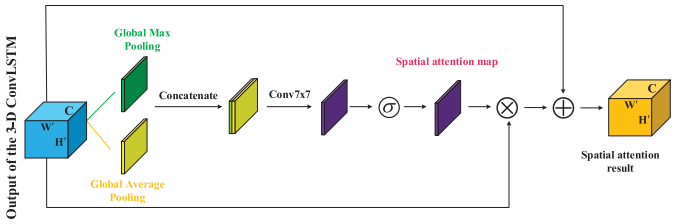


Fig. 5. Global spatial attention mechanism.

$h_8^{(2)} \in \mathcal{R}^{1\times 4\times 12\times H'\times W'}$, as the output of the 3-D ConvLSTM module, which can be formulated as

$$h_8^{(2)} = \text{Cell_2}\left(h_8^{(1)}, h_7^{(2)}, c_7^{(2)}\right). \tag{4}$$

Here, $h_8^{(2)}$ contains strong short-range and long-range cross-channel spectral dependencies. Finally, $h_8^{(2)}$ is concatenated at the channel dimension and used as the input of the GJAM module [55] to emphasize the importance of the extracted features.

GJAM has carefully designed two global attention mechanisms for the spatial domain and the spectral domain. The core idea is to use global maximum pooling and global average pooling to extract two different feature vectors, and then use these two feature vectors to calculate the spectral attention map and spatial attention map. Finally, using automatic broadcasting to perform a weighted product of the original features in the channel dimension and the spatial dimension. For simplicity, we show the calculation flowcharts of the global spectral attention mechanism and the global spatial attention mechanism in Figs. 4 and 5, respectively. The obtained weighted spectral features and spatial features are then concatenated at the channel dimension to recover the channel size (i.e., 96, 128, 192, and 256) of the input of the 3-D ConvLSTM module. Now, turn attention to the statistical loss.

B. Statistical Loss

Traditional losses mainly use the estimated posterior probability and the ground-truth label of the sample to calculate the loss and backpropagates to update the network parameters, such as the most popular and widely used cross entropy (softmax) loss. However, this type of loss only considers the information of a single pixel (training sample) in HSI and ignores the correlation between different training samples. Statistical loss assumes that the high-level abstract features of each class of training sample in HSI extracted by the network model obey a certain multivariate normal distribution, and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

uses the estimated probability distribution to formulate the loss function. In other words, it aims to decrease the variance of the features that belong to the same class while making the distance between features belonging to different categories as large as possible.

Suppose there is a p-dimensional random variable $V = (V_1, V_2, \ldots, V_p)$ that obeys a certain multivariate normal distribution, and its probability dense function (pdf) $f_V(\mathbf{v})$ can be written as

$$f_V(\mathbf{v}) = \frac{1}{(2\pi)^{\frac{p}{2}}|\Sigma|^{\frac{1}{2}}}\exp\left[-\frac{1}{2}(\mathbf{v}-\mu)^T\Sigma^{-1}(\mathbf{v}-\mu)\right] \quad (5)$$

where $\mathbf{v} = (v_1, v_2, \ldots, v_p)$, $-\infty < v_i < \infty (i = 1, 2, \ldots, p)$, μ denotes the mean vector of the normal distribution, and Σ denotes the covariance matrix of the normal distribution. For the convenience of description, we express this multivariate normal distribution as $V \sim N_p(\mu, \Sigma)$. Then, in the final feature space, the features extracted from each category sample $k (k = 1, 2, \ldots, \Lambda)$ in HSI can be regarded as a p-dimensional random variable, which obeys the multivariate normal distribution $N_p(\mu_k, \Sigma_k)$, in which p denotes the dimension of the highly abstract features obtained in the final convolutional feature space, and Λ represents the total number of object categories of interest that appear in HSI. Obviously, the feature variables extracted from different categories are independent of each other.

We introduce statistical loss not only to penalize the within-class variance of the acquired features of each class. Moreover, to learn more discriminative features, this work adds a simple but effective diversity-promoting term, which repulses different class distribution from each other. Therefore, from the statistical view, feature correlation of the training samples can be characterized with the probabilistic model. In summary, we give the complete formula for statistical loss as follows:

$$L_s = \arg\min_{\theta} \frac{1}{\Lambda}\sum_{k=1}^{\Lambda} tr(\Sigma_k) - \lambda\sum_{k\neq m}^{\Lambda}\|\mu_k - \mu_m\|_F \quad (6)$$

where tr$(\cdot)$ represents the operator that calculates the trace of the matrix, and θ represents the trainable parameters of the entire model in RSSGL; k and m represent different land covers from the image, $\|\cdot\|_F$ means the Frobenius norm, and λ is the diversity-promoting weight. An important point needs to be stated here, λ was set to 0.001 in our experiments to keep the variance term and the diversity-promoting term in the statistical loss at an order of magnitude, and this default value usually works well, so we do not recommend you to change it. Next, to calculate the statistical loss given in (6) during the training process, we need to estimate the mean vector μ and the covariance matrix Σ_k for each class.

In RSSGL, we use the H-B sampling strategy [55] to generate mini-batch masks, i.e., $M_1, M_2, \ldots, M_{10}$ (one epoch contains ten mini-batches), and then use M_i and the padded ground truth Y_{pad} to select training batch. In simple terms, every mini-batch contains $10 \times \Lambda$ training samples; that is, ten training samples are randomly chosen from each class to form a mini-batch. The H-B sampling strategy can get different error gradients to ensure the convergence of the FCN model working in RSSGL. But, it should be noted here that RSSGL is a global learning framework, the input of the training process is always the entire HSI, and we only use the selected training batch labels (i.e., $Y_i \in \mathcal{R}^{H'\times W'}$) to calculate the loss during backpropagation.

Given the mini-batch training samples B at hand. Let $\mathbf{x}_i$ denote any training sample in B, and $\mathbf{z}_i$ denote the feature of $\mathbf{x}_i$ learned by the model in the final feature space (as shown in Fig. 1). Also, let $X_k^B = \{\mathbf{x}_{s_1}, \mathbf{x}_{s_2}, \ldots, \mathbf{x}_{s_{n_k}}\}$ represent the set of kth class training samples in B, and $C_k^B = \{\mathbf{z}_{s_1}, \mathbf{z}_{s_2}, \ldots, \mathbf{z}_{s_{n_k}}\}$ denote the set of the learned features for these samples. Among them, n_k represents the number of the kth class training samples involved in B. The learned features in C_k^B obey the normal distribution $N_p(\mu_k, \Sigma_k)$, and the unbiased estimate $\bar{C}_k$ for the mean vector μ_k can be expressed as

$$\bar{C}_k = \frac{1}{n_k}\sum_{i=1}^{n_k}\mathbf{z}_{s_i}. \quad (7)$$

Besides, for such training samples, let $\hat{\Sigma}_k$ denote the unbiased estimate for the covariance matrix Σ_k, and it can be derived from the following formula:

$$\hat{\Sigma}_k = \frac{1}{n_k - 1}\sum_{j=1}^{n_k}(\mathbf{z}_j - \bar{C}_k)(\mathbf{z}_j - \bar{C}_k)^T \quad (8)$$

where T stands for transpose operation. Now, $\hat{\Sigma}_k$ can be used to approximate the covariance matrix Σ_k, and we have

$$tr(\hat{\Sigma}_k) = \frac{1}{n_k - 1}\sum_{j=1}^{n_k}(\mathbf{z}_j - \bar{C}_k)^T(\mathbf{z}_j - \bar{C}_k). \quad (9)$$

Thence, the statistical loss developed in (6) can be rewritten as

$$L_s = \arg\min_{\theta}\frac{1}{\Lambda}\sum_{k=1}^{\Lambda}\left[\frac{1}{n_k - 1}\sum_{j=1}^{n_k}(\mathbf{z}_j - \bar{C}_k)^T(\mathbf{z}_j - \bar{C}_k)\right] - \lambda\sum_{k\neq m}^{\Lambda}\|\mu_k - \mu_m\|_F. \quad (10)$$

It is worth emphasizing that the whole FCN model in RSSGL uses joint weighted cross entropy loss and statistical loss for training.

HSI classification is different from the natural image classification task. Although you can use the whole HSI, which contains training samples, testing samples, and the background to extract the spectral–spatial features, you are not allowed to use information, such as the label and quantity of the testing samples. From this perspective, the weighting factor for the kth class training samples in the weighted cross entropy used in SSDGL should be changed as

$$p_j = \frac{0.0001 \times \Lambda}{(1 - 0.9999^{N_j}) \times \sum_{k=1}^{\Lambda}(1 - 0.9999^{N_j})} \quad (11)$$

where N_j represents the number of the training samples of the jth class. As a summary, we give the loss function used to train the whole model as shown in the following:

$$L = L_{\text{wc}} + \alpha L_s \quad (12)$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

where α represents the weight of the statistical loss, and L_{wc} denotes the weighted cross entropy, which can be written as

$$L_{\mathrm{wc}} = -\frac{1}{\mathrm{sum}(M_i)} \sum p_{j(j\in Y_i)} \log(\hat{Y}_i[Y_i]) \tag{13}$$

where $\hat{Y}_i \in \mathcal{R}^{\Lambda\times H'\times W'}$ represents the output of the softmax layer.

C. Early Stopping

As shown in (12), the weighted cross entropy L_{wc} occupies the dominant position in the training loss function, and the statistical loss L_s is only used as a regularization term (deep metric loss) to restrict the model parameters θ. L_{wc} addresses the unbalanced sample problem and pays more attention to misclassified samples, especially categories with fewer training samples. However, even if the statistical loss as a deep metric regularization term is added, the dominant weight cross entropy is only a variant of cross entropy. When this loss is used as the objective function for the HSI classification problem under a global learning framework with unbalanced and limited training samples, although the H-B sampling strategy proposed in SSDGL can obtain diverse gradients in backpropagation, it still makes the model unable to converge smoothly. This is mainly because the number of training samples of some categories is too small, and the categories with a relatively large number of training samples have a certain degree of randomness when using the H-B sampling strategy to construct mini-batches, which makes the training process oscillate toward the end. In addition, when training a large model with sufficient representation ability or even overfitting, it can often be observed that the training error will gradually decrease over time but the validation set error will rise again. This means that as long as we return to the parameter setting that minimizes the validation set error, we can get a model with a lower validation set error and hopefully get a better test error.

To solve the abovementioned problems, we introduce the early stopping strategy into RSSGL, which may be the most commonly used regularization form in deep learning. Its popularity is mainly due to its effectiveness and simplicity. In RSSGL, we use a list to collect the overall classification accuracy [overall accuracy (OA)] of the model on the validation set after each epoch to store the best model parameters and set the early stopping counter c and patience P two variables to control when the training process stops. This strategy is described more formally in Algorithm 1.

Algorithm 1 Early Stopping

Input: A list for collecting validation set OA: V; Early stopping counter: c; Patience: P; Training epochs: E; Epoch index: j.
Output: The best model parameters θ^*.

```
Initialize V = [0.], c = 0;
while j < E do
    Update θ;
    V.append(ValidationSetOA(θ));
    if V[−2] > V[−1] then
        if c = 0 then
            θ* ← θ;
        end
        c ← c + 1;
        V[−1] ← V[−2];
        if c = P then
            θ ← θ*;
            print(Early Stopped! The best OA: V[−1]);
            raise StopIteration('Early Stopping!');
        end
    else
        c ← 0;
    end
    j ← j + 1
end
reture θ*
```

III. Experimental Results and Analysis

To analyze the effectiveness of our proposed framework quantitatively and qualitatively, rich classification experiments were carried out on three well-known real HSI datasets: the 16-class IP dataset,[1] the 9-class PU dataset, and the 16-class SA scene dataset. These datasets are used in case of unbalanced samples data, high spatial resolution data, and regularly spatial distributed data. The performance is assessed in terms of OA, which calculates the ratio of correctly classified samples to all samples in the test set, average accuracy (AA), which is a common metric used to determine the average of class-wise accuracy and evaluate how well the classification results are for classes with fewer training samples, and statistical Kappa (κ) coefficient calculated based on the confusion matrix indicates the degree of agreement between the classification map obtained by the classification model and the ground truth. We compared some of the most advanced deep models proposed in the past two years, including A2S2K-ResNet [26], HybridSN [27], SpectralNET [30], miniGCN (FuNet-C) [34], FPGA [51], and SSDGL [55]. All experiments were performed on a system with NVIDIA GeForce RTX 3090 24-GB GPU and 32-GB RAM. To remove the impact of completely randomly selected training samples on the classification results, each model was run five times on each dataset, and each evaluation metric was present in the form of mean value $\pm$ standard deviation.

A. Experimental Setup

1) *Model Parameters:* The model used in RSSGL is an FCN based on encoder and decoder architecture. Each gated unit in the 3-D ConvLSTMCell (i.e., Cell_1 and Cell_2) used four 3-D convolution kernels to perform input-to-state and state-to-state convolution operations, and the kernels' shape was set to $3\times5\times5$. In particular, due to the IP dataset has a small spatial size, in the

[1] http://www.ehu.eus/ccwintco/uploads/6/67/Indian_pines_corrected.mat, http://www.ehu.eus/ccwintco/uploads/e/ee/PaviaU.mat, https://github.com/gokriznastic/HybridSN/raw/master/data/Salinas_corrected.mat

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

TABLE II
EXPERIMENTAL CONFIGURATIONS ON DIFFERENT DATASETS

Parameters	IP	SA	PU
3-D kernel shape	$3 \times 3 \times 3$	$3 \times 5 \times 5$	$3 \times 5 \times 5$
weight decay	0.001	0.001	0.001
η_0	0.005	0.005	0.005
E	100	60	60
P	15	10	10
α	0.0009	0.001	0.005

experiments on this dataset, we set the 3-D convolution kernels' shape to $3\times3\times3$. The number of groups for GN was set to 4, and the number of channels for the output feature map of each downsampling convolutional layer was set to 128, 192, and 256, respectively. Moreover, this framework preserves the lateral connection used in FPGA and SSDGL, which not only facilitates the exchange and fusion of information between the encoder and the decoder, but also helps reuse the lower level and more realistic features learned by the encoder, and the output channel of the lateral connections was set 128. Finally, the output channel of the final 1×1 convolutional layer before the softmax layer corresponds to the total class number Λ. Other default parameters are shown in Fig. 1 and will not be described in detail here.

2) *Optimizer and Learning Rate Schedule:* In the training process of the deep CNN model, the optimizer will not only affect the speed of training, but may also affect the performance. Stochastic gradient descent with momentum optimization was used as the optimizer of RSSGL, where the base learning rate was set to 0.005, and the momentum value was set to 0.9. In addition, the weight decay (L2 penalty) coefficient, which prevents the model from overfitting, was set to 0.001. We hope that the learning rate will continue to decrease as the training process progresses; therefore, the power function $\eta_0(1 - \text{iter}/\text{max_iter})^{\text{power}}$ was adopted as the learning rate schedule, where power $= 0.8$, η_0 represents the base learning rate, iter denotes the iteration number (i.e., one mini-batch corresponds to one iteration, and ten iterations correspond to one epoch), and max_iter $=$ 1000. Finally, considering that different datasets have different spatial sizes, the training epochs on IP, SA, and PU datasets were set to 100, 60, and 60, respectively.

3) *Other Hyperparameters:* The early stopping patience P in Algorithm 1 was set to 15, 10, and 10, respectively, for experiments on IP, PU, and SA datasets to avoid the uncertainty and randomness of the training process. As for the statistical loss weight α in (12), the values set on different datasets are different: 0.0009 for IP, 0.001 for SA, and 0.005 for PU.

For clarity, Table II lists the abovementioned experimental configurations on the different datasets.

B. Experiment 1: IP Dataset

This scene was collected by airborne visible/infrared imaging spectrometer (AVIRIS) at the IP test site in northwestern India. It consists of 145×145 pixels and 224 spectral bands with a wavelength range of 0.4–2.5 μm. Due to water absorption and corruption, 24 spectral bands in which have been removed. The referenced ground truth is designed as 16 vegetation categories and not all are mutually exclusive, some of which have highly unbalanced sample sizes. The spatial resolution of this dataset is approximately 20 m per pixel. Problems, such as low resolution, spectral confounding, and unbalanced samples, have brought great challenges to classification task on this dataset. Fig. 6(a) and (b) shows the three-band false color image (bands 29, 19, and 9) of this dataset and the color-coded image of the ground truth.

For this dataset, we randomly selected 5% of labeled samples as training samples. If the number of training samples for a certain class is less than 5, by default, at least five labeled samples are selected as training samples, and each mini-batch Y_i always contains these five training samples of this class. It should be noted that, because of the rapid inference ability of the global learning framework, FPGA, SSDGL, and RSSGL do not have separate validation sets, so data other than training data are used as test data to evaluate accuracy.

The average OAs, AAs, κ, and per-class accuracies obtained using A2S2K-ResNet, HybridSN, SpectralNET, miniGCN, FPGA, SSDGL, and the proposed RSSGL are reported in Table III. The best values for each row in Table III are highlighted in bold. To observe the classification results more intuitively, the visualization of classification maps generated by different methods (when these methods obtained the best OA on the test set in five runs) is shown in Fig. 6(c)–(i).

As shown in Table III, since the global learning framework performs feature extraction in the entire spatial domain, the global spatial context information can be fully explored, and patch-free methods (FPGA, SSDGL, and the proposed RSSGL) obtained higher OA, AA, and κ values than those patch-based methods (A2S2K-ResNet, HybridSN, SpectralNET, and FuNet-C) and, thus, have better generalization ability. Although there are only five training samples for class 9 (Oats) and class 16 (Stone-Steel-Towers), RSSGL obtained higher accuracy (recall ratio) than other methods in these two categories and achieved competitive accuracy on Alfalfa (class 1). Apart from this, RSSGL also obtained the highest AA value (97.80%) than other methods, which shows the effectiveness of the weighted cross entropy as it imposes a greater penalty when the model misclassifies a category with a small support set of training samples. It is worth noting that A2S2K-ResNet achieved the highest classification accuracy on class 5 (Grass-pasture) as well as the highest OA value and Kappa coefficient in patch-based methods, which verifies that the attention-based adaptive spectral–spatial kernel is capable of extracting discriminative spectral–spatial joint features. In addition, even though FuNet-C has the lowest overall classification accuracy (84.73%), the classification accuracy on Oats (class 9) still reaches 100%, which is encouraging. Interestingly, both FPGA and SSDGL achieved an OA value of 97.12% and a Kappa coefficient of 0.9672, but the standard deviation of SSDGL on these two metrics is relatively low. However, the AA value obtained by SSDGL is lower than the one obtained by FPGA, and the standard deviation is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

TABLE III

OA, AA, AND κ VALUES OBTAINED BY APPLYING DIFFERENT METHODS TO THE IP DATASET WITH 5% TRAINING SAMPLES

	class	Patch-based				Patch-free		
		A2S2K-ResNet	HybridSN	SpectralNET	miniGCN (FuNet-C)	FPGA	SSDGL	Proposed
Accuracy(%)	C1	97.99 ± 2.918	91.70 ± 9.949	95.12 ± 6.360	59.51 ± 7.956	97.56 ± 1.543	**99.02 ± 1.195**	98.05 ± 3.902
	C2	94.23 ± 2.172	95.74 ± 1.697	90.21 ± 1.506	79.59 ± 2.041	95.16 ± 3.963	**96.12 ± 2.520**	95.91 ± 0.647
	C3	93.31 ± 1.836	90.18 ± 3.519	92.13 ± 3.624	78.71 ± 9.719	96.70 ± 1.855	96.40 ± 1.242	**97.69 ± 1.659**
	C4	96.38 ± 3.595	93.33 ± 3.229	87.11 ± 5.774	62.49 ± 12.843	**98.76 ± 1.625**	95.20 ± 4.690	96.09 ± 2.731
	C5	**98.43 ± 1.429**	90.26 ± 6.350	90.09 ± 3.492	90.66 ± 0.639	94.80 ± 4.579	94.50 ± 3.838	93.89 ± 3.857
	C6	98.49 ± 1.114	96.91 ± 2.138	95.30 ± 2.960	98.38 ± 0.853	**99.22 ± 0.414**	98.27 ± 1.313	98.18 ± 1.553
	C7	82.60 ± 7.813	93.91 ± 4.434	94.78 ± 8.431	91.30 ± 7.778	99.13 ± 1.739	96.52 ± 6.957	**100.00 ± 0.000**
	C8	98.90 ± 1.248	**100.00 ± 0.000**	97.58 ± 2.882	99.74 ± 0.257	99.91 ± 0.108	**100.00 ± 0.000**	**100.00 ± 0.000**
	C9	73.54 ± 12.615	98.67 ± 2.667	89.33 ± 15.549	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**
	C10	93.20 ± 2.808	92.46 ± 3.182	91.35 ± 3.351	79.52 ± 3.808	93.67 ± 3.711	**97.05 ± 1.465**	94.99 ± 0.770
	C11	95.73 ± 2.009	95.87 ± 1.680	96.92 ± 2.151	87.80 ± 3.160	**98.74 ± 0.770**	97.04 ± 0.945	97.78 ± 0.655
	C12	94.77 ± 2.173	87.07 ± 6.527	84.12 ± 4.933	66.22 ± 5.323	92.22 ± 4.470	93.32 ± 3.241	**96.52 ± 1.104**
	C13	97.74 ± 2.059	94.74 ± 9.750	96.70 ± 2.364	99.59 ± 0.206	**99.69 ± 0.412**	98.97 ± 1.815	99.48 ± 0.799
	C14	97.92 ± 2.044	98.50 ± 0.936	96.64 ± 1.289	94.37 ± 1.823	99.00 ± 1.133	**99.98 ± 0.033**	99.67 ± 0.508
	C15	92.66 ± 4.017	89.23 ± 7.393	92.90 ± 5.964	60.82 ± 3.544	97.92 ± 2.895	97.81 ± 2.279	**97.98 ± 1.690**
	C16	87.95 ± 4.872	87.73 ± 7.343	80.91 ± 8.486	93.41 ± 1.818	97.05 ± 2.928	93.86 ± 7.350	**98.63 ± 1.325**
	OA (%)	95.48 ± 0.852	94.46 ± 0.609	93.33 ± 0.679	84.73 ± 0.794	97.12 ± 0.674	97.12 ± 0.464	**97.38 ± 0.350**
	AA (%)	93.37 ± 1.422	93.52 ± 0.595	91.95 ± 0.951	83.88 ± 1.746	97.47 ± 0.607	97.13 ± 0.851	**97.80 ± 0.390**
	κ	0.9484 ± 0.010	0.9368 ± 0.007	0.9239 ± 0.008	0.8255 ± 0.009	0.9672 ± 0.008	0.9672 ± 0.005	**0.9701 ± 0.004**

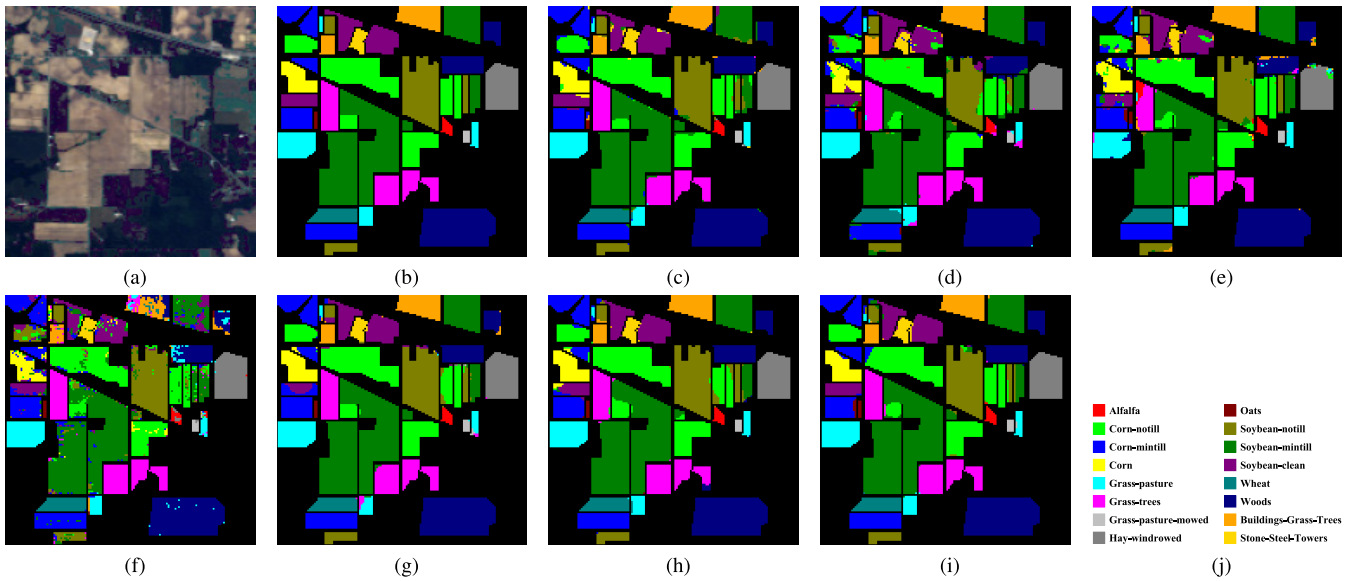


Fig. 6. Visual comparison charts for IP dataset. (a) Three-band false color image. (b) Ground truth. (c)–(i) Classification maps predicted by A2S2K-ResNet, HybridSN, SpectralNET, miniGCN (FuNet-C), FPGA, SSDGL, and the proposed RSSGL, respectively. (j) Color legend.

higher. This is mainly because the weighted cross entropy used in SSDGL is a kind of sample-based loss, which will cause instability in the training process. Finally, the statistical loss was added to the weighted cross entropy to improve the feature extraction ability of the FCN model in RSSGL, and the early stopping strategy is adopted to prevent RSSGL from oscillating. Therefore, the proposed method achieves better classification results in the case of insufficient and unbalanced samples. It can be seen that compared with FPGA and SSDGL, RSSGL has higher values and lower standard deviations on OA, AA, and Kappa coefficient.

C. Experiment 2: SA Dataset

To further evaluate the performance of the proposed RSSGL on a dataset with more regularly distributed ground objects, this subsection introduces the experiments conducted on the SA dataset. This dataset was gathered by AVIRIS sensor (224-band) over SA Valley in California, and it has a high spatial resolution of 3.7 m. The covered area includes 512 rows by 217 columns of samples. As with IP scenario, 20 water absorption and noise bands have been discarded. SA ground truth also contains 16 categories of interest. Fig. 7(a) and (b) shows the three-band false color image and the color-coded

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
OA, AA, AND κ VALUES OBTAINED BY APPLYING DIFFERENT METHODS TO THE SA DATASET WITH 1% TRAINING SAMPLES

class		Patch-based				Patch-free		
		A2S2K-ResNet	HybridSN	SpectralNET	miniGCN (FuNet-C)	FPGA	SSDGL	Proposed
Accuracy(%)	C1	**100.00 ± 0.000**	99.91 ± 0.157	99.90 ± 0.071	94.60 ± 3.084	99.85 ± 0.302	99.98 ± 0.040	99.42 ± 1.142
	C2	99.85 ± 0.232	**100.00 ± 0.000**	99.67 ± 0.379	97.83 ± 2.056	99.86 ± 0.271	99.80 ± 0.388	99.69 ± 0.435
	C3	99.47 ± 0.702	99.93 ± 0.143	97.24 ± 1.346	93.25 ± 2.733	**100.00 ± 0.000**	99.84 ± 0.302	99.97 ± 0.061
	C4	98.04 ± 1.487	97.84 ± 2.529	95.94 ± 5.952	99.52 ± 0.345	**99.91 ± 0.071**	99.84 ± 0.148	99.23 ± 1.157
	C5	98.11 ± 1.596	**99.45 ± 0.500**	98.44 ± 1.838	98.64 ± 0.754	99.09 ± 0.235	99.11 ± 0.522	98.82 ± 0.763
	C6	99.86 ± 0.166	99.70 ± 0.579	99.35 ± 0.756	**99.99 ± 0.010**	99.91 ± 0.184	99.95 ± 0.069	99.92 ± 0.102
	C7	99.45 ± 0.594	99.98 ± 0.045	99.19 ± 0.345	99.59 ± 0.052	**99.99 ± 0.011**	99.98 ± 0.023	99.99 ± 0.023
	C8	89.42 ± 3.590	99.22 ± 0.765	91.85 ± 2.453	78.40 ± 6.122	98.78 ± 0.949	98.54 ± 1.322	**99.83 ± 0.096**
	C9	99.75 ± 0.212	99.97 ± 0.065	99.33 ± 0.554	99.28 ± 0.544	**100.00 ± 0.000**	**100.00 ± 0.000**	99.92 ± 0.078
	C10	97.61 ± 2.259	98.59 ± 0.904	98.61 ± 1.100	93.39 ± 1.615	**99.67 ± 0.223**	99.62 ± 0.180	98.89 ± 1.122
	C11	96.43 ± 2.288	99.26 ± 1.032	96.05 ± 3.632	84.24 ± 7.785	99.02 ± 1.098	**99.72 ± 0.169**	94.61 ± 3.238
	C12	97.94 ± 1.600	99.71 ± 0.408	96.87 ± 2.579	95.70 ± 4.089	**99.86 ± 0.247**	99.70 ± 0.355	99.30 ± 0.941
	C13	94.70 ± 6.572	99.71 ± 0.367	93.62 ± 5.729	98.85 ± 0.492	**99.96 ± 0.054**	97.57 ± 1.760	99.38 ± 0.434
	C14	97.67 ± 1.600	99.77 ± 0.366	98.51 ± 1.847	93.96 ± 2.005	**99.79 ± 0.234**	99.30 ± 0.457	98.94 ± 0.639
	C15	94.54 ± 2.727	97.91 ± 2.067	93.00 ± 3.865	77.99 ± 7.845	99.84 ± 0.258	99.80 ± 0.366	**99.87 ± 0.106**
	C16	99.56 ± 0.737	99.94 ± 0.112	98.50 ± 1.506	97.45 ± 0.769	97.60 ± 2.916	98.99 ± 1.086	**100.00 ± 0.000**
OA (%)		96.20 ± 0.671	99.32 ± 0.327	96.41 ± 0.313	90.67 ± 0.805	99.53 ± 0.182	99.47 ± 0.277	**99.58 ± 0.105**
AA (%)		97.65 ± 0.556	99.43 ± 0.191	97.25 ± 0.351	93.92 ± 0.473	**99.57 ± 0.125**	99.48 ± 0.168	99.24 ± 0.304
κ		0.9576 ± 0.008	0.9924 ± 0.004	0.9601 ± 0.003	0.8964 ± 0.009	0.9947 ± 0.002	0.9941 ± 0.003	**0.9953 ± 0.001**

image of the relevant ground truth. We set the percentage of training samples for each category to 1% of the labeled samples.

For this dataset, the number of training samples is much smaller than the number of test samples. Since the training samples in each experiment are randomly selected, this will have different effects on the final classification results. Table IV lists the classification accuracy obtained by different compared methods, where it can be seen that RSSGL performs much better than other methods in OA and Kappa. However, RSSGL achieved a relatively low AA value and high deviation, which is mainly due to the random selection of training samples. This phenomenon does not mean that the weighted cross entropy used in SSDGL and RSSGL is incorrect. To eliminate the influence of random training samples on the experimental results, we will give a fair comparison of the experimental results of FPGA, SSDGL, and RSSGL methods in the case of fixed training samples in Section IV.

We also observe that the expected value obtained by FPGA in OA is close to RSSGL. We speculate that this is mainly due to the simple and effective spectral attention module in FPGA, which can provide more spectral information when the spatial information is saturated, making it more suitable for the SA dataset. The classification accuracy on class 8 (Grapes-untrained) using RSSGL achieved a 1.05% improvement over FPGA and achieved a 2.4% improvement on class 16 (Vinyard-vertical-trellis). Moreover, RSSGL obtained the highest accuracy on class 15 (Vinyard-untrained). The number of test samples for class 8 (Grapes-untrained) and class 15 (Vinyard-untrained) accounted for the vast majority of the total number of test samples. This is why the AA value obtained by RSSGL is lower than the one obtained by FPGA and SSDGL, but the highest OA value is achieved. RSSGL benefits from the 3-D ConvLSTM module and statistical loss, which enables this framework to extract the most discriminative spectral–spatial features. Furthermore, considering the complexity of the model, the early stopping strategy plays a very important role in the training process. Finally, among the patch-based methods, HybridSN performs much better than others, and we speculate that this is because HybridSN uses three 3-D CNN layers and each layer has multiple convolution kernels, which allows it to extract deeper, more abstract and more easily distinguishable features.

Fig. 7 shows the visual performance of different methods when they achieved the highest OA value in five runs. It can be found that the classification maps in Fig. 7(g) (FPGA) and Fig. 7(i) (SSDGL) are better than those in Fig. 7(c)–(f). FPGA contains fewer misclassified noise points in the obtained classification map, and the result of the proposed RSSGL is slightly worse than FPGA, SSDGL, and even HybridSN. The main reason for this consequence is that the selection of training samples is random. The highest OA value obtained by RSSGL in the experiments is lower than the one obtained by FPGA, SSDGL, and HybridSN. For a fair comparison, we will also give a comparison of the classification maps in the case of fixed training samples in Section IV. Nevertheless, RSSGL has a good classification effect for Grapes-untrained (class 8), which indicates that for the HSI classification task, the global 3-D ConvLSTM can more fully extract combined spectral–spatial information.

D. Experiment 3: PU Dataset

This scene was acquired by the reflective optics system imaging spectrometer (ROSIS) sensor during a flight activity over the PU in 2001. Among them, there are 12 bands

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

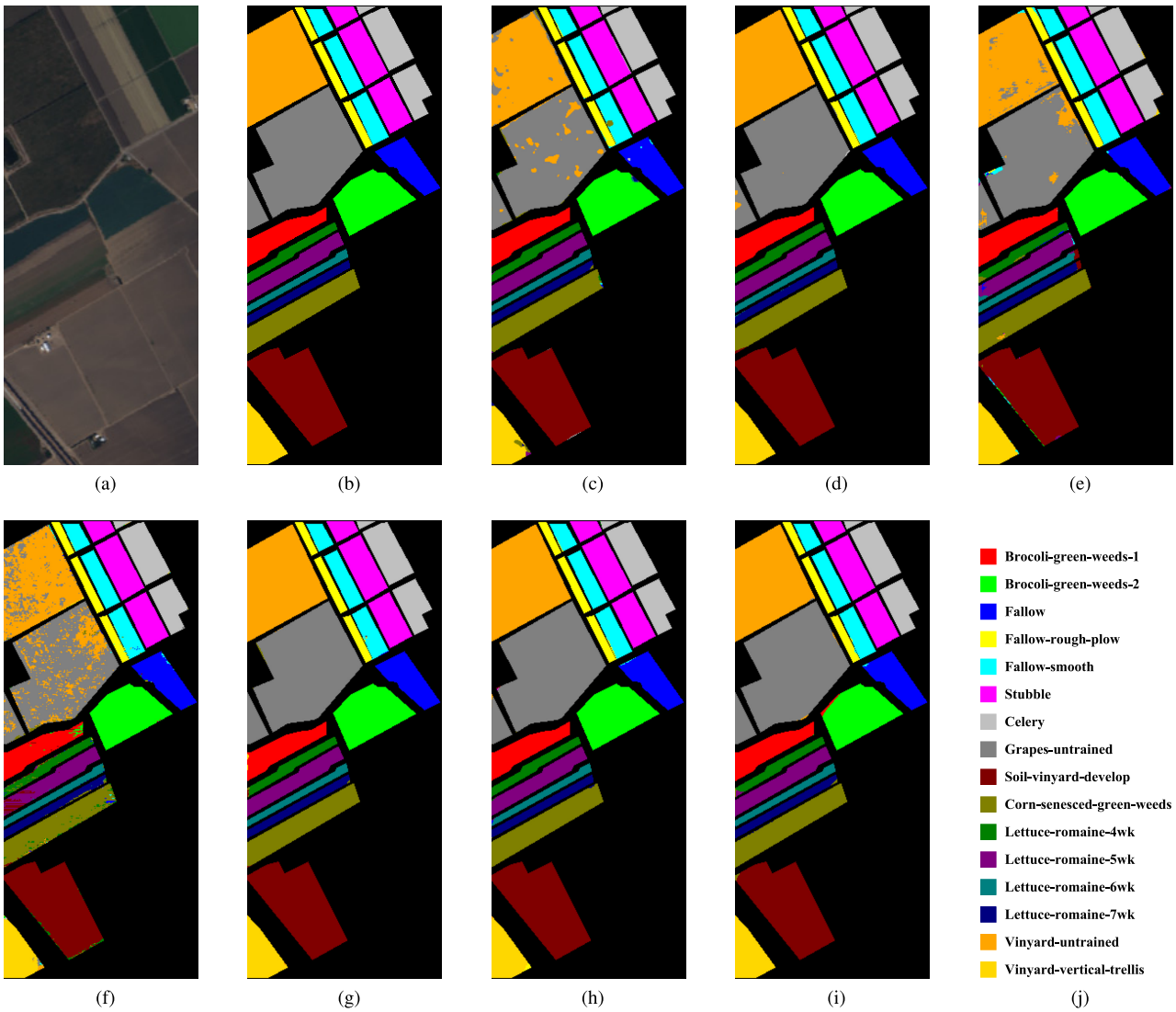


Fig. 7. Visual comparison charts for SA dataset. (a) Three-band false color image. (b) Ground truth. (c)–(i) Classification maps predicted by A2S2K-ResNet, HybridSN, SpectralNET, miniGCN (FuNet-C), FPGA, SSDGL, and the proposed RSSGL, respectively. (j) Color legend.

that cover noise and water absorption areas and have been discarded, so only 103 bands of data are retained, and the wavelength ranges from 0.43 to 0.86 μm. This dataset presents the ground scene in a 610×340 image format, with a spatial resolution of 1.3 m per pixel, and accommodates nine types of urban land covers. Fig. 8(a) and (b) shows the three-band false color image and the color-coded image of the ground truth, as a visual reference. In this set of experiments, we also selected 1% of the labeled samples for each category as the training samples.

Table V shows the OA, AA, and Kappa values obtained by all the compared methods on this dataset, which makes it easier to quantitatively evaluate the performance of different methods on a dataset with such a high spatial resolution. By the way, on high spatial resolution datasets, spatial information becomes more important for categories that are difficult to distinguish. As we can see from Table V, although patch-free methods have higher classification accuracy than patch-based methods as a whole, FPGA and SSDGL are very sensitive to the training samples of gravel (class 3). In their respective five experiments, the mean classification accuracies on gravel are 90.06% and 88.00%, respectively. But, both FPGA and SSDGL have very high deviations (8.390 and 8.099, respectively). The proposed RSSGL achieved the highest accuracy on gravel with the lowest deviation among all the compared methods. This result reflects that the proposed framework is not only effective but also stable. In addition, in five experiments, the classification accuracy of Metal Sheets (class 5) and Bare Soil (class 6) is 100%; in the case of randomly selecting training samples, this result is unattainable by other methods. Among the patch-based methods, HybridSN still achieved the highest overall classification accuracy, which indicates the effectiveness of the cascaded multilevel 3-D convolution layers in extracting spectral–spatial joint features. Besides that, it is worth noting that miniGCN also obtained competitive classification results, even better than SpectralNET, which can be concluded that nonlocal spatial information plays a key role in the classification task for high spatial resolution remote

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

TABLE V

OA, AA, AND κ VALUES OBTAINED BY APPLYING DIFFERENT METHODS TO THE PU DATASET WITH 1% TRAINING SAMPLES

class		Patch-based				Patch-free		
		A2S2K-ResNet	HybridSN	SpectralNET	miniGCN (FuNet-C)	FPGA	SSDGL	Proposed
Accuracy(%)	C1	91.83 ± 2.690	96.14 ± 1.720	94.34 ± 1.328	91.76 ± 0.408	98.46 ± 0.431	96.05 ± 1.387	**99.57 ± 0.332**
	C2	97.17 ± 1.274	99.73 ± 0.225	97.76 ± 0.586	99.47 ± 0.206	**99.87 ± 0.116**	99.80 ± 0.186	99.86 ± 0.125
	C3	86.55 ± 3.730	92.28 ± 4.004	65.89 ± 8.475	72.02 ± 5.399	90.06 ± 8.390	88.00 ± 8.099	**97.57 ± 1.635**
	C4	**98.75 ± 1.066**	89.79 ± 2.811	95.34 ± 1.700	95.57 ± 1.319	97.07 ± 1.023	96.48 ± 2.053	98.23 ± 0.596
	C5	99.91 ± 0.181	99.85 ± 0.233	99.70 ± 0.297	99.59 ± 0.287	99.92 ± 0.116	99.71 ± 0.334	**100.00 ± 0.000**
	C6	97.73 ± 1.558	99.85 ± 0.145	93.73 ± 2.838	89.11 ± 1.775	100.00 ± 0.008	99.78 ± 0.266	**100.00 ± 0.000**
	C7	93.76 ± 5.421	**98.63 ± 1.660**	71.22 ± 8.156	79.92 ± 2.462	96.23 ± 4.619	94.48 ± 5.145	97.84 ± 1.446
	C8	81.72 ± 3.597	90.92 ± 4.112	91.95 ± 3.034	87.09 ± 2.869	**99.20 ± 0.473**	98.39 ± 0.714	97.74 ± 1.848
	C9	95.27 ± 4.399	76.63 ± 10.324	89.07 ± 4.861	**99.59 ± 0.207**	98.87 ± 0.677	98.12 ± 1.114	98.59 ± 0.455
OA (%)		94.49 ± 0.731	96.81 ± 0.648	93.56 ± 0.562	93.76 ± 0.256	98.79 ± 0.325	98.07 ± 0.271	**99.34 ± 0.305**
AA (%)		93.63 ± 1.391	93.76 ± 1.472	88.78 ± 1.222	90.46 ± 0.608	97.74 ± 0.907	96.76 ± 0.533	**98.82 ± 0.576**
κ		0.9265 ± 0.010	0.9576 ± 0.009	0.9146 ± 0.007	0.9169 ± 0.003	0.9840 ± 0.004	0.9745 ± 0.004	**0.9912 ± 0.004**

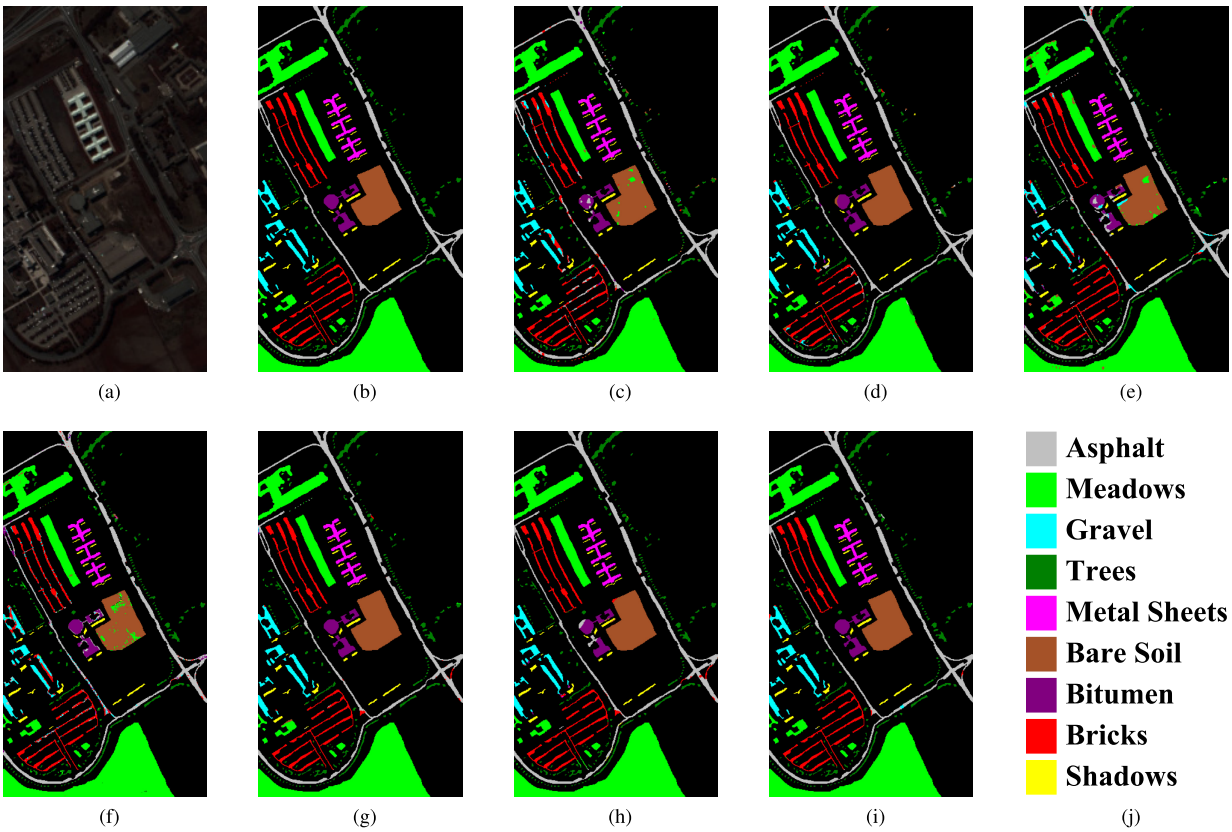


Fig. 8. Visual comparison charts for PU dataset. (a) Three-band false color image. (b) Ground truth. (c)–(i) Classification maps predicted by A2S2K-ResNet, HybridSN, SpectralNET, miniGCN (FuNet-C), FPGA, SSDGL, and the proposed RSSGL, respectively. (j) Color legend.

sensing images. Finally, attributed to the learning of long-range and short-range cross-channel spectral dependencies and the statistical loss regularization, RSSGL achieved unparalleled high values in OA, AA, and Kappa coefficient.

The visual performance of the classifier may often be more important for HSI classification tasks. Fig. 8(c)–(i) shows the final classification charts obtained by all the compared methods when they achieved the highest OA values in five runs. As can be seen in Fig. 8(c)–(i), patch-free classifiers have better visual performance than the patch-based classifiers, and this is mainly because the global learning frameworks facilitate the extraction of global spatial features and, thus, improve the classification performance. Among the different methods, the classification map obtained by RSSGL contains the least noise points, which reflects that RSSGL can capture finer spatial detail.

IV. DISCUSSION

To further analyze the effectiveness of the proposed RSSGL framework, we conducted a series of extensible experiments in this section.

A. Use of Fixed Training Samples

In this subsection, we randomly selected training samples and conducted five experiments on each dataset to examine the average performance of each method. As mentioned earlier,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

OA, AA, AND κ VALUES OBTAINED BY APPLYING DIFFERENT METHODS TO THE IP AND SA DATASETS USING FIXED 5% AND 1% TRAINING SAMPLES (SEED = 2333)

class		IP			SA			
		FPGA	SSDGL	Proposed	HybridSN	FPGA	SSDGL	Proposed
Accuracy(%)	C1	87.80 ± 0.000	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	99.08 ± 1.117	**100.00 ± 0.000**
	C2	93.57 ± 0.055	95.88 ± 0.304	**96.31 ± 0.536**	**100.00 ± 0.000**	99.94 ± 0.074	99.94 ± 0.085	99.99 ± 0.022
	C3	90.23 ± 0.000	**92.97 ± 0.492**	92.53 ± 0.629	99.99 ± 0.020	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**
	C4	**100.00 ± 0.000**	90.13 ± 1.300	96.18 ± 0.665	99.96 ± 0.087	99.94 ± 0.054	99.93 ± 0.092	**100.00 ± 0.000**
	C5	**95.85 ± 0.000**	94.32 ± 0.138	93.71 ± 0.321	99.62 ± 0.245	99.48 ± 0.250	99.72 ± 0.130	**99.77 ± 0.055**
	C6	97.98 ± 0.091	**98.82 ± 0.212**	97.69 ± 0.129	99.91 ± 0.066	99.99 ± 0.010	**100.00 ± 0.000**	**100.00 ± 0.000**
	C7	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	99.95 ± 0.068	99.99 ± 0.011	99.96 ± 0.079	**100.00 ± 0.000**
	C8	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	99.08 ± 0.499	99.70 ± 0.161	99.69 ± 0.171	**99.71 ± 0.093**
	C9	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	99.99 ± 0.013
	C10	97.12 ± 0.053	**99.00 ± 0.081**	96.88 ± 0.408	99.05 ± 0.520	99.83 ± 0.127	99.86 ± 0.106	**99.88 ± 0.188**
	C11	**98.04 ± 0.021**	95.20 ± 0.264	96.33 ± 0.166	99.92 ± 0.093	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**
	C12	92.72 ± 0.112	92.36 ± 0.225	**93.61 ± 0.195**	99.39 ± 0.682	**99.85 ± 0.121**	98.01 ± 0.833	98.07 ± 0.659
	C13	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**	99.60 ± 0.561	**100.00 ± 0.000**	99.98 ± 0.044	99.89 ± 0.171
	C14	**100.00 ± 0.000**	99.85 ± 0.082	99.68 ± 0.097	96.24 ± 3.163	99.49 ± 0.319	**99.91 ± 0.189**	99.75 ± 0.128
	C15	99.18 ± 0.000	**100.00 ± 0.000**	**100.00 ± 0.000**	98.68 ± 0.975	99.79 ± 0.175	99.07 ± 0.305	**99.86 ± 0.084**
	C16	96.59 ± 0.000	97.50 ± 0.455	**98.64 ± 0.850**	98.24 ± 3.027	98.11 ± 0.905	95.63 ± 3.413	**99.77 ± 0.160**
OA (%)		96.69 ± 0.023	96.54 ± 0.109	**96.73 ± 0.095**	99.38 ± 0.136	99.79 ± 0.045	99.53 ± 0.130	**99.81 ± 0.040**
AA (%)		96.82 ± 0.016	97.25 ± 0.116	**97.60 ± 0.111**	99.35 ± 0.214	99.76 ± 0.057	99.42 ± 0.247	**99.79 ± 0.053**
κ		0.9623 ± 0.000	0.9607 ± 0.001	**0.9628 ± 0.001**	0.9931 ± 0.002	0.9976 ± 0.001	0.9947 ± 0.001	**0.9980 ± 0.000**

random training samples will bring greater randomness to the classification results. In this part, we will select fixed training samples by setting a fixed random seed (SEED = 2333) and will also conduct five experiments on each dataset to mainly evaluate the performance of FPGA, SSDGL, and RSSGL as they all belong to the global learning framework. The reason for running five experiments to get the average classification accuracy is to eliminate the influence of the difference in weight initialization of each layer on the experimental results. By this way, we could have a fair comparison between different methods. Due to both IP and SA datasets contain 16 land-cover types, the classification results on IP and SA dataset using the same fixed training samples are reported in Table VI, and the classification results on PU dataset are reported in Table VII.

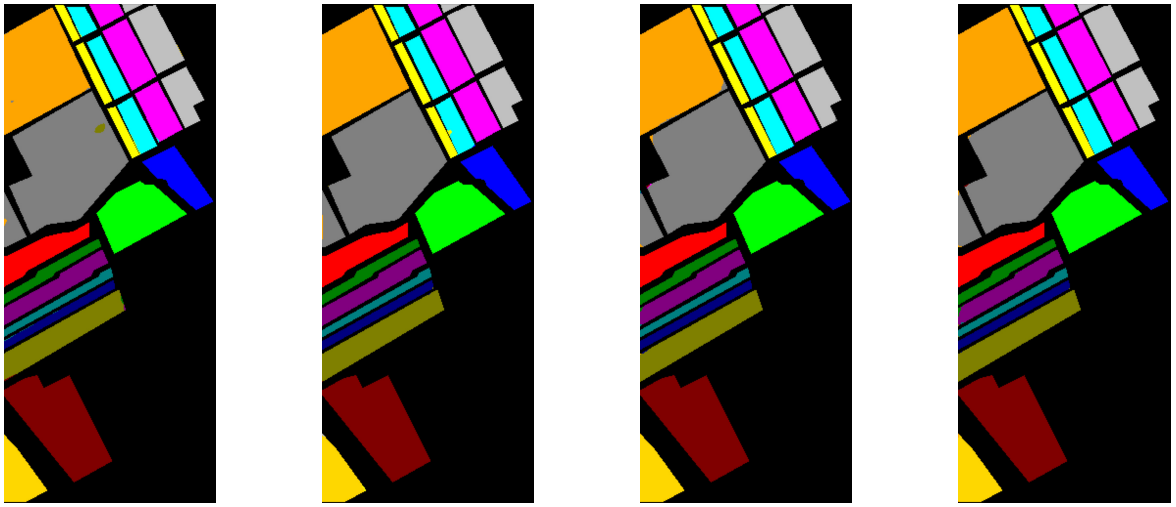
It can be seen from Table VI that when we use the same training samples, the OA values obtained by the proposed RSSGL on the IP and SA datasets are slightly higher than the ones obtained by FPGA. Although the improvement under these fixed training samples is not obvious, it is enough to show that the proposed framework can also obtain competitive results. An interesting phenomenon is that although the OA value obtained by SSDGL is slightly lower than the value obtained by FPGA on the IP dataset, the AA value obtained by it is 0.43% higher than the AA value obtained by FPGA, and this also fully shows that SSDGL pays more attention to the classification performance of categories with fewer training samples. However, on the SA dataset, when the OA values obtained by these two methods differ greatly, SSDGL cannot guarantee that a higher AA value will be obtained. Besides that, the AA values achieved by RSSGL on the IP and SA datasets are both the highest, which shows that the weighted cross entropy has a certain contribution to improve the average classification accuracy. Finally, by observing Table VII, when the same training samples are used, the proposed method has the overwhelming advantages in all three metrics. Compared with the IP and SA datasets, the performance improvement on the PU dataset using RSSGL is the most obvious. We conjecture that when the spatial resolution of HSI is high, whether it is a global learning framework or a GCN model, when the spatial context information cannot provide more help to the classification performance, the high-level abstract spectral features will be more beneficial. In other words, because RSSGL extracts long-range and short-range cross-channel spectral dependence information through the 3-D ConvLSTM

TABLE VII

OA, AA, AND κ VALUES OBTAINED BY APPLYING DIFFERENT METHODS TO THE PU DATASET USING FIXED 1% TRAINING SAMPLES (SEED = 2333)

class		PU		
		FPGA	SSDGL	Proposed
Accuracy(%)	C1	98.69 ± 0.287	96.30 ± 1.621	**99.06 ± 0.403**
	C2	**99.94 ± 0.064**	99.65 ± 0.320	99.89 ± 0.122
	C3	96.31 ± 0.926	93.22 ± 3.462	**98.63 ± 0.614**
	C4	97.01 ± 0.890	96.47 ± 0.778	**97.57 ± 0.373**
	C5	**100.00 ± 0.000**	99.89 ± 0.131	**100.00 ± 0.000**
	C6	**100.00 ± 0.000**	**100.00 ± 0.000**	**100.00 ± 0.000**
	C7	**99.57 ± 0.776**	95.70 ± 3.006	99.24 ± 0.913
	C8	98.95 ± 0.780	98.66 ± 0.644	**99.72 ± 0.284**
	C9	**98.63 ± 1.120**	97.55 ± 1.824	97.95 ± 1.650
OA (%)		99.24 ± 0.126	98.38 ± 0.419	**99.47 ± 0.080**
AA (%)		98.79 ± 0.150	97.49 ± 0.848	**99.12 ± 0.252**
κ		0.9899 ± 0.002	0.9786 ± 0.006	**0.9930 ± 0.001**

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 9. Visual comparison charts for SA dataset with the fixed training samples (SEED = 2333). (a) HybridSN. (b) FPGA. (c) SSDGL. (d) RSSGL.

module, this makes it a better classification effect for HSI with high spatial resolution. Therefore, the proposed RSSGL framework is trustworthy. Here, we have also fulfilled the promise mentioned earlier. On the SA dataset, due to the randomness of the training samples, in a random experiment, HybridSN obtained even better classification performance than RSSGL. Therefore, we not only give the classification performance of HybridSN in the case of fixed training samples in Table VI, but also re-compared the optimal classification maps obtained by each method in Fig. 9. As can be seen from Fig. 9, the optimal classification maps obtained by RSSGL and FPGA are obviously better than SSDGL and HybridSN.

B. Effects of the Statistical Loss Weight

As described in Section II-B, α represents the statistical loss weight (i.e., regularizer coefficient), which is an important hyperparameter in RSSGL framework. The value of α will undoubtedly affect the performance of the proposed framework. Therefore, a set of special experiments is needed to evaluate the impact of different α values on RSSGL performance. Due to the small spatial size of the IP dataset and the shortest training time, we will conduct a series of comparative experiments on this dataset. Here, we choose the value of α from the set of {0.0008, 0.0009, 0.001, 0.002, 0.003}. In practical applications, training samples are usually given, so we still select fixed training samples by setting a random seed global variable (SEED = 2333). Fig. 10 shows the performance of the framework when using different values of α over the IP dataset.

By observing the error bar graph, it can be found that the statistical loss regularizer can provide better performance through appropriate α, and larger or smaller values will have varying degrees of impact on performance. When increasing the value of α, it will increase the constraints on the model parameters, which will help reduce the variance of the model, but increase its deviation. While reducing its value, although the model capacity can be increased, it also makes the prediction of the model more extreme and unreasonable. By observing Fig. 10, when α was set to 0.002 over the IP dataset, the performance of the model on the test set is also acceptable. Therefore, in practical applications, since the available training samples are limited and unbalanced and the test set is untouchable, we recommend to perform a grid search within the value range of 0.001–0.005 in K-fold cross validation to select an optimal regularizer coefficient. This may take a considerable amount of time, but after completion, you can get the best hyperparameter value.

C. Comparison of Different Loss Functions

To further confirm the powerfulness of the statistical loss applied in RSSGL, we performed ablation studies while unchanging other experimental settings (as described in Section III-A). Since RSSGL has the most obvious performance improvement on the PU dataset, all comparative experiments were performed on this dataset. In addition, we still choose fixed training samples to compare various methods after ablation.

1) The statistical loss is added to FPGA and keeps the cross entropy unchanged (the resulting method is denoted as FPGA + SL).
2) The statistical loss is added to SSDGL and keeps the weighted cross entropy unchanged (the resulting method is denoted as SSDGL + SL).
3) The statistical loss is removed from RSSGL and keeps the weighted cross entropy unchanged [the resulting method is denoted as RSSGL-SL(WC)].
4) The statistical loss is removed from RSSGL, and the weighted cross entropy is substituted by the normal cross entropy [the resulting method is denoted as RSSGL-SL(CE)].
5) The weighted cross entropy is substituted by the normal cross entropy and keeps the statistical loss unchanged [the resulting method is denoted as RSSGL(CE + SL)].
6) The statistical loss is substituted by the IT loss and keeps the weighted cross entropy unchanged [the resulting method is denoted as RSSGL(WC + IT)].

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

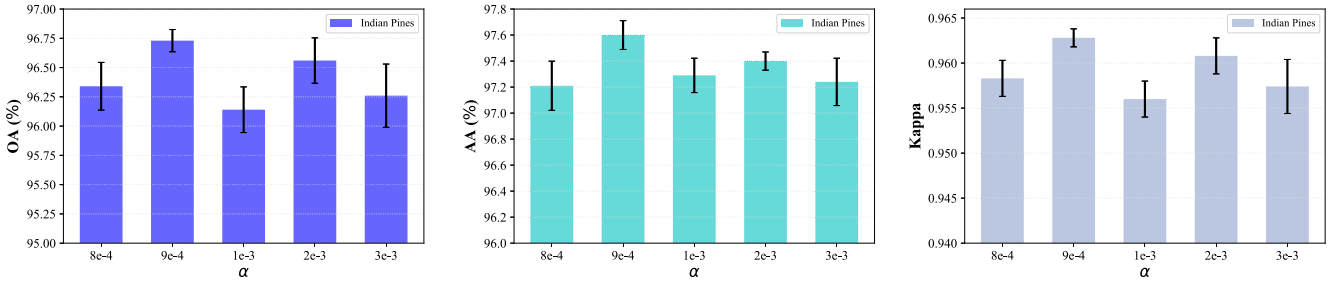


Fig. 10. Classification performance of the proposed RSSGL with different statistical loss weight α over IP dataset. (a) Impact on the OA of different α settings. (b) Impact on the AA of different α settings. (c) Impact on the Kappa of different α settings.

TABLE VIII

ACCURACY ANALYSIS FOR DIFFERENT ABLATION METHODS ON THE PU DATASET WITH FIXED 1% TRAINING SAMPLES (SEED = 2333)

class		PU						
		FPGA+SL	SSDGL+SL	RSSGL-SL(WC)	RSSGL-SL(CE)	RSSGL(CE+SL)	RSSGL(WC+IT)	RSSGL(WC+SP)
Accuracy(%)	C1	**98.80 ± 0.348**	98.08 ± 0.581	97.87 ± 0.515	98.15 ± 0.352	98.63 ± 0.701	98.46 ± 0.780	98.27 ± 0.826
	C2	**99.99 ± 0.008**	99.98 ± 0.036	99.70 ± 0.091	99.95 ± 0.017	99.97 ± 0.034	99.67 ± 0.061	99.73 ± 0.133
	C3	97.14 ± 0.376	98.25 ± 0.974	96.48 ± 1.542	98.56 ± 0.301	98.65 ± 0.696	96.83 ± 3.031	**98.85 ± 0.642**
	C4	97.29 ± 0.697	**98.15 ± 0.632**	96.21 ± 0.565	97.61 ± 0.490	96.70 ± 1.414	97.78 ± 0.433	97.57 ± 0.717
	C5	**100.00 ± 0.000**	**100.00 ± 0.000**	98.82 ± 0.169	99.97 ± 0.037	**100.00 ± 0.000**	99.40 ± 0.525	99.97 ± 0.060
	C6	**100.00 ± 0.000**	99.99 ± 0.016	99.92 ± 0.143	99.97 ± 0.064	**100.00 ± 0.000**	99.98 ± 0.032	**100.00 ± 0.000**
	C7	**99.85 ± 0.304**	99.10 ± 0.833	98.71 ± 0.880	98.30 ± 1.791	99.77 ± 0.192	97.90 ± 1.351	99.48 ± 0.530
	C8	99.82 ± 0.088	99.69 ± 0.304	99.73 ± 0.157	99.61 ± 0.243	**99.85 ± 0.123**	99.60 ± 0.308	99.73 ± 0.317
	C9	98.91 ± 0.529	98.95 ± 0.605	98.14 ± 0.792	97.97 ± 0.801	98.68 ± 0.451	97.31 ± 2.190	**99.66 ± 0.347**
OA (%)		**99.43 ± 0.067**	99.39 ± 0.108	98.98 ± 0.100	99.31 ± 0.102	99.42 ± 0.151	99.12 ± 0.187	99.34 ± 0.094
AA (%)		99.09 ± 0.121	99.13 ± 0.142	98.51 ± 0.190	98.90 ± 0.290	99.14 ± 0.231	98.55 ± 0.501	**99.25 ± 0.045**
κ		**0.9925 ± 0.001**	0.9920 ± 0.001	0.9864 ± 0.001	0.9909 ± 0.001	0.9924 ± 0.002	0.9884 ± 0.002	0.9912 ± 0.001

7) The statistical loss is substituted by the sphere loss and keeps the weighted cross entropy unchanged [the resulting method is denoted as RSSGL(WC + SP)].

When comparing the performance of RSSGL(WC + IT) and RSSGL(WC + SP), it is necessary to make the following statements.

1) The margin parameter (see [47]) included in the IT loss was set to 0.2, and the top four hardest (positive and negative) pairs were selected to calculate the intraclass and interclass distances. The parameter $\tan^2 \beta$ appeared in the sphere loss was set 0.2 (see [48]).
2) Both the IT loss and the sphere loss appeared in the loss function as regularization terms [see (12)] and the regularization weights (see [48]) were both set to 0.001.

Table VIII gives the classification results of different ablation methods using fixed training samples over the PU dataset. By observing Tables VII and VIII, compared with FPGA and SSDGL, both FPGA + SL and SSDGL + SL provide a significant improvement in terms of OA, AA, and Kappa. Another point worth noting is that the average AA value obtained by SSDGL + SL is even higher than RSSGL, which once again proves that the contribution of the weighted cross entropy to the AA metric is worthy of recognition. In addition, although the AA value obtained by FPGA + SL is still lower than the one obtained by RSSGL, FPGA + SL achieves comparable performance in terms of OA and Kappa. We also observe that, when we remove the statistical loss from RSSGL, i.e., RSSGL-SL(WC), its performance drops significantly. This is mainly because only the weighted cross entropy is used as the loss function of RSSGL-SL(WC), which is the same as the loss function of SSDGL. Nevertheless, RSSGL-SL(WC) still outperforms SSDGL in all indicators, which also reflects the importance of fully exploiting spectral dependencies. Meanwhile, when using the normal cross entropy as the loss function, namely, RSSGL-SL(CE), which has the same loss function as FPGA, its performance still surpasses FPGA. By comparing SSDGL, RSSGL-SL(WC), and RSSGL-SL(CE), we can see that the models do not perform well when using the weighted cross entropy developed in SSDGL alone as the loss function. But, to the best of our knowledge, few studies emphasize the importance of the AA metric, and from the performance of SSDGL + SL and RSSGL, we still insist on preserving the weighted cross entropy.

Next, let us briefly analyze RSSGL(CE + SL). When we replace the weighted cross entropy in (12) with ordinary cross entropy, i.e., RSSGL(CE + SL), the expected values it obtained on OA and Kappa are close to FPGA + SL, and the expected value obtained on AA is slightly higher than RSSGL. Also, the maximum values that RSSGL(CE + SL) could obtain on the three metrics are close to the ones that RSSGL could obtain. For the reason that, we do not exclude

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

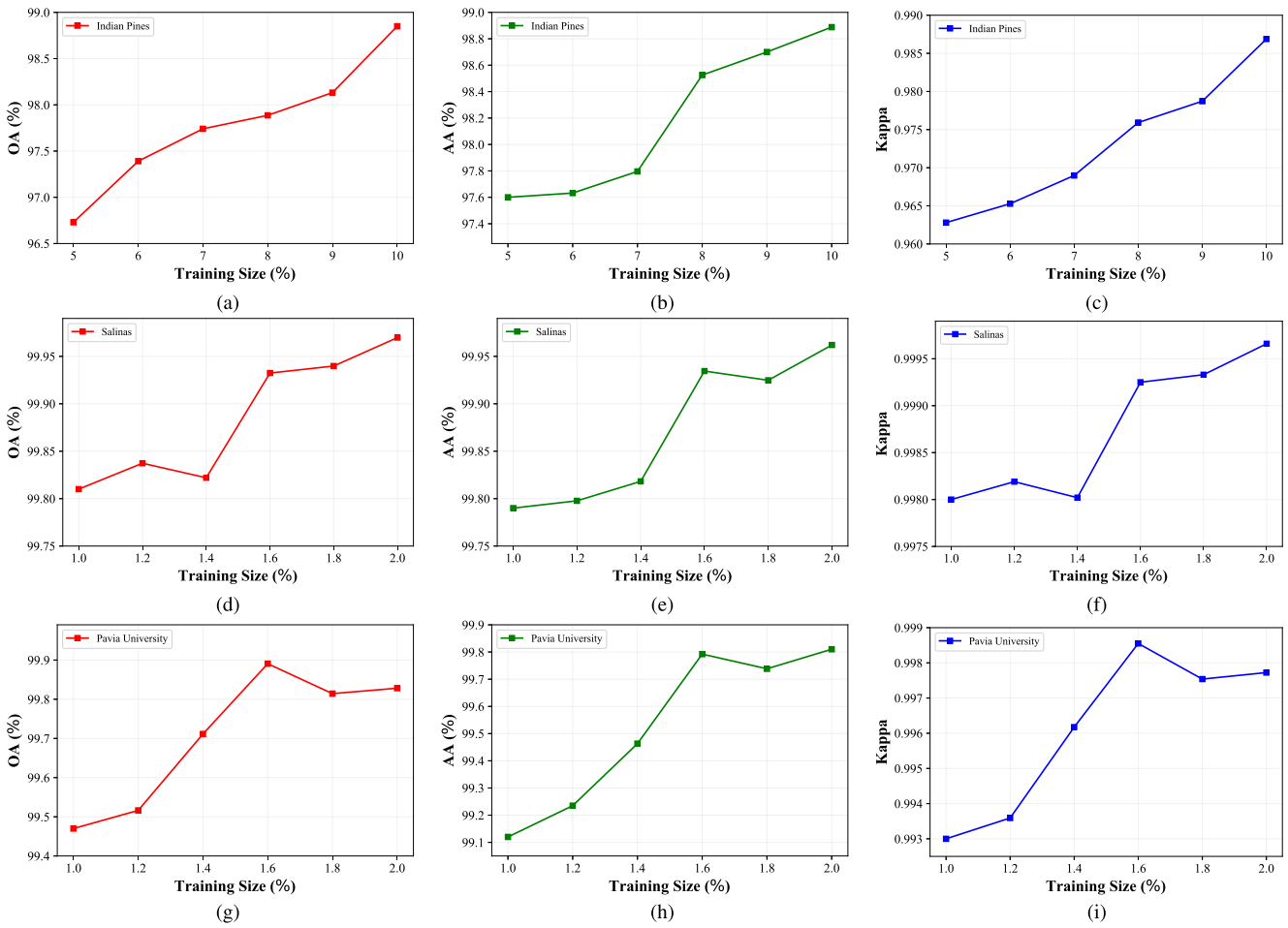


Fig. 11. Performance under different proportions of training samples. (a)–(i) Performance on IP, SA, and PU datasets, respectively.

the use of ordinary cross entropy as the main body of the loss function, but ordinary cross entropy does not emphasize the problem of unbalanced classification; that is, it does not explicitly emphasize the importance of AA.

At last, by comparing RSSGL, RSSGL(WC + IT), and RSSGL(WC + SP), we can find that the IT loss and the sphere loss do not improve the proposed framework as much as the statistical loss. Although RSSGL(WC + SP) obtained the highest expected value on AA, this is mainly due to the use of the weighted cross entropy. Hence, we can conclude that the statistical loss plays a key role in the proposed framework as it performs deep metric learning in the embedded feature space from the perspective of probability distribution. Therefore, the extracted features are more discriminative and make it easier for the model to distinguish different objects.

D. Effects of Different Proportions of Training Samples

Section IV-C has proved the validity of the statistical loss regularizer developed. What this subsection will evaluate is the performance of the proposed framework with different training sample ratios. For the IP dataset, we select 6%–10% training samples for each class from the entire data. While for the PU and SA datasets, we select the percentage of training samples for each class from the set of {1.2%, 1.4%, 1.6%, 1.8%, 2%}. We randomly choose training samples from the given ground truth by setting a fixed random seed (SEED = 2333), and other samples were used as testing samples to evaluate classification performance. All experiments were run only once, although there are errors, it is also possible to intuitively understand the impact of different proportions of training samples on the framework. The results are plotted in Fig. 11.

In general, increasing the training samples will improve the performance of RSSGL. For the IP dataset, as the proportion of training samples per class continues to increase, all metrics are steadily improved. But, for the SA and PU datasets, although we only conducted one experiment with different proportions of training samples, we can observe some interesting phenomena. For the SA dataset, when we select 1.4% training samples per class, the OA value obtained is slightly lower than the value obtained when we select 1.2% training samples per class, but the AA and Kappa values are slightly higher than the latter. However, when we select 1.8% training samples for each class, the OA and Kappa values obtained are slightly higher than those obtained when we select 1.6% training samples for each class, but the situation of the AA value is just the opposite. A similar phenomenon can also be observed on the PU dataset when we select 1.6% and 2% training samples per

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
COMPARISON OF THE COMPUTATIONAL COSTS OF DIFFERENT PATCH-FREE FRAMEWORKS ON THE THREE DATASETS

Data Sets	Frameworks	Trainable Parameters	GPU Memory (MB)	Cost (10^9×FLOPs)	Training Time (m)
IP	FPGA	2.648M	2145	15.054	0.82
	SSDGL	2.266M	7465	28.038	1.82
	RSSGL	1.738M	2237	34.194	8.1
SA	FPGA	2.652M	5969	67.839	1.37
	SSDGL	2.274M	6163	126.543	3.67
	RSSGL	1.786M	5997	319.006	20.43
PU	FPGA	2.564M	7241	110.558	1.87
	SSDGL	2.068M	7413	197.092	5.8
	RSSGL	1.580M	10523	565.693	44.39

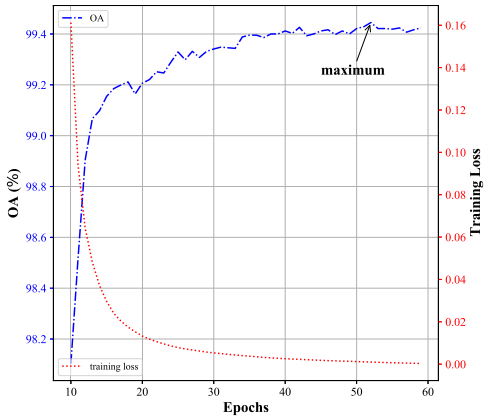


Fig. 12. Training loss and OA values over epochs on the PU dataset.

class. We speculate that the main reason for the inconsistency of these three metrics is the introduction of the weighted cross entropy. Therefore, although a high AA value may lead to a lower OA value, the training samples in HSI classification are usually unbalanced, and we hope to maintain this balance between AA and OA, instead of pursuing a higher OA value blindly.

E. Necessity of Early Stopping

Although early stopping is common and popular today, it has a non-negligible role in a global learning framework for solving classification problems with few and unbalanced samples. Since global learning extracts spectral–spatial joint features over the entire dataset, to accelerate the convergence speed, it is better to design a reasonable mini-batch sampling strategy. Using the H-B sampling strategy to construct mini-batches is a good approach. However, since the training samples are few and unbalanced, even with a reasonable learning rate schedule, this sampling strategy still cannot guarantee that the model will see sufficiently diverse training samples within several epochs close to convergence. To better observe the impact of the H-B sampling strategy on model convergence, Fig. 12 shows the training loss and OA values over epochs on the PU dataset.

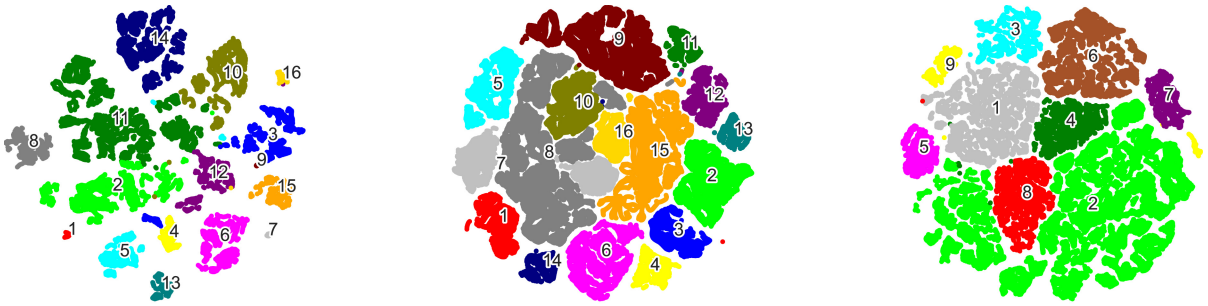
From the picture, we can see that as the training progress, the training loss continues to decrease. Besides, the overall classification accuracy is already higher than 99.4% at the 40th epoch and achieves the best performance at the 52nd epoch. However, from the last 20 epochs, the performance of the model is constantly oscillating, which not only carries the risk of overfitting, but also wastes time. Therefore, the introduction of early stopping can not only effectively shorten the training time but also restore the optimal model, which is a very important and neglected point for the global learning framework.

F. Computational Cost and t-SNE Visualization

Experimental results on different datasets show that the classification results of patch-free methods are significantly better than patch-based methods. So, here, we mainly do complexity analysis on FPGA, SSDGL, and RSSGL. Table IX shows the model complexity of FPGA, SSDGL, and RSSGL (on three benchmark datasets) relative to the amount of trainable weight parameters renewed during back propagation, GPU memory footprint, floating-point operations (FLOPs), and average training time. It can be observed that the number of trainable parameters used by the proposed framework is the least, whereas FPGA requires the most parameters. In which, Table IX also lists the memory requirements of each framework. It is clear that when the spatial size of HSI is small (i.e., IP and SA datasets), the GPU memory used by the proposed framework is close to that used by FPGA. However, on account of the existence of 3-D ConvLSTMCells, RSSGL needs to take up more GPU memory when the spatial size of HSI is large. The number of floating-point operations can be more straightforward to express the computational cost, and its unit is expressed as 10^9 × FLOPs. As can be seen from Table IX, the proposed framework has the highest FLOPs over three datasets, which is mainly due to the use of 3-D convolution in the ConvLSTMCell. The value of FLOPs will directly affect the training time of the framework.

Intuitively, in five experiments on the PU dataset, our proposed framework used an average training time of 44.39 min, which takes the highest training time, whereas FPGA only takes 1.87 min. Although the early stopping strategy is adopted to effectively shorten the training time (the average training time saved by the early stopping strategy relative to the set epochs on the IP, SA, and PU datasets is 50 s, 7 min 29 s, and 10 min 6 s, respectively), this is indeed a thorny issue when

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 13. 2-D t-SNE visualization of the extracted features from IP, SA, and PU datasets. Data distribution of the labeled samples in the final convolutional feature space (please refer to the mark in Fig. 1). (a) IP. (b) SA. (c) PU. (Best viewed in color.)

the spatial size of HSI is large, or the computing capacity of the graphic card is low. But, the good thing is that for HSI with high spatial resolution, the proposed RSSGL has a significant improvement, and it is acceptable to sacrifice model complexity in exchange for higher accuracy. Based on the abovementioned analysis, as a summary, we give the following suggestions.

1) For scenarios with high timeliness requirements, FPGA + SL can ensure both speed and accuracy.
2) If the training speed and average recall rate (AA) need to be considered at the same time, SSDGL + SL is more respected.
3) Given sufficient training time, the proposed RSSGL is worth the first choice anyway.

Finally, when using fixed training samples (i.e., 5%, 1%, and 1% for IP, SA, and PU datasets, respectively), to demonstrate the abstract learning ability of our proposed framework on different datasets, we use the 2-D t-distributed stochastic neighbor embedding (t-SNE) algorithm to visualize the features extracted in the final convolutional space. As shown in Fig. 13, the features extracted from the same class are close to each other and clustered into one point cloud, while the features extracted from different classes are separated from each other. Thus, dedicating to reduce the intraclass variance and increase the interclass distance in the final feature space, RSSGL has the most discriminative representation ability even when the training samples are limited and unbalanced.

V. Conclusion

In this work, we have presented a statistical loss regularized 3-D ConvLSTM global learning framework (RSSGL) for HSI supervised classification tasks. To maximize the mining of the spectral information and the global spatial context details, the long-range and short-range cross-channel spectral dependencies and spatial features are jointly extracted by the employed 3-D ConvLSTM. To further extract more discriminative features, based on the statistical analysis, we developed an improved statistical loss regularizer, which contains a variance minimization term and a diversity-promoting term to make the intraclass variance as small as possible and the interclass distance as large as possible in the final feature space. In addition, an early stopping strategy is adopted to avoid the instability and randomness of the training process caused by the H-B sampling strategy. The classification performance was evaluated on the IP dataset with high spectral confounding, the SA dataset with regular distribution of ground objects, and the PU dataset with high spatial resolution. Compared with other the most recent and, therefore, considered the best methods, RSSGL has obtained competitive or even better classification results. In particular, when the spatial resolution of HSI is high, and it may not be possible to use any means to use spatial information to continue to improve the classification performance, RSSGL can make fuller use of spectral information and can learn more easily distinguishable spectral–spatial joint features.

The introduction of 3-D convolution makes the model more complex, which is a problem that we will focus on in future work. Besides, it will be fun to try to get to the bottom of the performance of the proposed framework on HSI semi-supervised and unsupervised classification tasks.

References

[1] L. He, J. Li, C. Liu, and S. Li, "Recent advances on spectral-spatial hyperspectral image classification: An overview and new guidelines," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1579–1597, Mar. 2017.

[2] P. Ghamisi, J. Plaza, Y. Chen, J. Li, and A. J. Plaza, "Advanced spectral classifiers for hyperspectral images: A review," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 1, pp. 8–32, Mar. 2017.

[3] Y. Luo, J. Zou, C. Yao, X. Zhao, T. Li, and G. Bai, "HSI-CNN: A novel convolution neural network for hyperspectral image," in *Proc. Int. Conf. Audio, Lang. Image Process. (ICALIP)*, Jul. 2018, pp. 464–469.

[4] Y. Cai, X. Liu, and Z. Cai, "BS-Nets: An end-to-end framework for band selection of hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 3, pp. 1969–1984, Mar. 2020.

[5] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019.

[6] Z. Liu, L. Ma, and Q. Du, "Class-wise distribution adaptation for unsupervised classification of hyperspectral remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 508–521, Jan. 2021.

[7] M. Song, X. Shang, and C.-I. Chang, "3-D receiver operating characteristic analysis for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 11, pp. 8093–8115, Nov. 2020.

[8] Y. Chen, N. M. Nasrabadi, and T. D. Tran, "Hyperspectral image classification using dictionary-based sparse representation," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 10, pp. 3973–3985, Oct. 2011.

[9] J. A. Benediktsson, J. A. Palmason, and J. R. Sveinsson, "Classification of hyperspectral data from urban areas based on extended morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 480–491, Mar. 2005.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

[10] P. Duan, P. Ghamisi, X. Kang, B. Rasti, S. Li, and R. Gloaguen, "Fusion of dual spatial information for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7726–7738, Sep. 2021.

[11] H. Zhang, C. Gong, Y. Bai, Z. Bai, and Y. Li, "3-D-ANAS: 3-D asymmetric neural architecture search for fast hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, no. 99, pp. 1–19, May 2021.

[12] R. Hang, Z. Li, Q. Liu, P. Ghamisi, and S. S. Bhattacharyya, "Hyperspectral image classification with attention aided CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2281–2293, Mar. 2021.

[13] B. Rasti *et al.*, "Feature extraction for hyperspectral imagery: The evolution from shallow to deep (overview and toolbox)," *IEEE Geosci. Remote Sens. Mag.*, vol. 8, no. 4, pp. 60–88, Dec. 2020.

[14] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798–1828, Aug. 2013.

[15] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[16] A. Santara *et al.*, "Bass Net: Band-adaptive spectral-spatial feature learning neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 9, pp. 5293–5301, Sep. 2017.

[17] N. Audebert, B. L. Saux, and S. Lefevre, "Deep learning for classification of hyperspectral data: A comparative review," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 159–173, Jun. 2019.

[18] X. Li, M. Ding, and A. Pižurica, "Deep feature fusion via two-stream convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2615–2629, Apr. 2020.

[19] B. Xi *et al.*, "Deep prototypical networks with hybrid residual attention for hyperspectral image classification," *IEEE J. Sel. Topics Appl., Earth Observ. Remote Sens.*, vol. 13, no. 99, pp. 3683–3700, Jun. 2020.

[20] L. Fang, G. Liu, S. Li, P. Ghamisi, and J. A. Benediktsson, "Hyperspectral image classification with squeeze multibias network," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 3, pp. 1291–1301, Mar. 2019.

[21] Y. Chen, L. Zhu, P. Ghamisi, X. Jia, G. Li, and L. Tang, "Hyperspectral images classification with Gabor filtering and convolutional neural network," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 12, pp. 2355–2359, Dec. 2017.

[22] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.

[23] X. Cao, F. Zhou, L. Xu, D. Meng, Z. Xu, and J. Paisley, "Hyperspectral image classification with Markov random fields and a convolutional neural network," *IEEE Trans. Image Process.*, vol. 27, no. 5, pp. 2354–2367, May 2018.

[24] X. Cao, J. Yao, Z. Xu, and D. Meng, "Hyperspectral image classification with convolutional neural network and active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 7, pp. 4604–4616, Jul. 2020.

[25] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 844–853, Feb. 2017.

[26] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2021.

[27] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[28] B. Xi, J. Li, Y. Li, R. Song, W. Sun, and Q. Du, "Multiscale context-aware ensemble deep KELM for efficient hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5114–5130, Jun. 2021.

[29] B. Xi *et al.*, "Multi-direction networks with attentional spectral prior for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5500915.

[30] T. Chakraborty and U. Trehan, "SpectralNET: Exploring spatial–spectral WaveletCNN for hyperspectral image classification," 2021, *arXiv:2104.00341*.

[31] X. Tang *et al.*, "Hyperspectral image classification based on 3-D octave convolution with spatial–spectral attention network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2430–2447, Mar. 2021.

[32] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2020.

[33] P. Sellars, A. I. Aviles-Rivero, and C.-B. Schonlieb, "Superpixel contracted graph-based learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 6, pp. 4180–4193, Jun. 2020.

[34] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[35] W. Song, S. Li, L. Fang, and T. Lu, "Hyperspectral image classification with deep feature fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 6, pp. 3173–3184, Jun. 2018.

[36] M. E. Paoletti *et al.*, "Capsule networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2145–2160, Apr. 2019.

[37] S. K. Roy, J. M. Haut, M. E. Paoletti, S. R. Dubey, and A. Plaza, "Generative adversarial minority oversampling for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5500615.

[38] R. Hang, F. Zhou, Q. Liu, and P. Ghamisi, "Classification of hyperspectral images via multitask generative adversarial networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 2, pp. 1424–1436, Feb. 2020.

[39] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.

[40] R. Hang, Q. Liu, D. Hong, and P. Ghamisi, "Cascaded recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5384–5394, Aug. 2019.

[41] S. Hao, W. Wang, Y. Ye, E. Li, and L. Bruzzone, "A deep network architecture for super-resolution-aided hyperspectral image classification with classwise loss," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4650–4663, Aug. 2018.

[42] S. Hao, W. Wang, and M. Salzmann, "Geometry-aware deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2448–2460, Mar. 2021.

[43] Z. Gong, P. Zhong, Y. Yu, W. Hu, and S. Li, "A CNN with multiscale convolution and diversified metric for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 6, pp. 3599–3618, Jun. 2019.

[44] B. Liu, X. Yu, P. Zhang, A. Yu, Q. Fu, and X. Wei, "Supervised deep feature extraction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 1909–1921, Nov. 2017.

[45] Z. Gong, W. Hu, X. Du, P. Zhong, and P. Hu, "Deep manifold embedding for hyperspectral image classification," *IEEE Trans. Cybern.*, early access, Apr. 19, 2021, doi: 10.1109/TCYB.2021.3069790.

[46] A. J. X. Guo and F. Zhu, "Spectral-spatial feature extraction and classification by ANN supervised with center loss in hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 3, pp. 1755–1767, Mar. 2019.

[47] K.-K. Huang, C.-X. Ren, H. Liu, Z.-R. Lai, Y.-F. Yu, and D.-Q. Dai, "Hyperspectral image classification via discriminative convolutional neural network with an improved triplet loss," *Pattern Recognit.*, vol. 112, Apr. 2021, Art. no. 107744.

[48] J. Wang, H. Chen, L. Ma, L. Chen, X. Gong, and W. Liu, "Sphere loss: Learning discriminative features for scene classification in a hyperspherical feature space," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5601819.

[49] J. Wang, F. Zhou, S. Wen, X. Liu, and Y. Lin, "Deep metric learning with angular loss," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2612–2620.

[50] Z. Gong, P. Zhong, and W. Hu, "Statistical loss and analysis for deep learning in hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 322–333, Jan. 2021.

[51] Z. Zheng, Y. Zhong, A. Ma, and L. Zhang, "FPGA: Fast patch-free global learning framework for fully end-to-end hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5612–5626, Aug. 2020.

[52] W. Qi, X. Zhang, N. Wang, M. Zhang, and Y. Cen, "A spectral-spatial cascaded 3-D convolutional neural network with a convolutional long short-term memory network for hyperspectral image classification," *Remote Sens.*, vol. 11, no. 20, p. 2363, Oct. 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.

[53] W.-S. Hu, H.-C. Li, L. Pan, W. Li, R. Tao, and Q. Du, "Spatial–spectral feature extraction via deep ConvLSTM neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 6, pp. 4237–4250, Jun. 2020.

[54] W.-S. Hu, H.-C. Li, Y.-J. Deng, X. Sun, Q. Du, and A. Plaza, "Lightweight tensor attention-driven ConvLSTM neural network for hyperspectral image classification," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 734–745, Apr. 2021.

[55] Q. Zhu *et al.*, "A spectral-spatial dependent global learning framework for insufficient and imbalanced hyperspectral image classification," *IEEE Trans. Cybern.*, early access, May 25, 2021, doi: 10.1109/TCYB.2021.3070577.

[56] W. Li, J. Yin, B. Han, and H. Zhu, "Generative adversarial network with folded spectrum for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2019, pp. 883–886.

[57] L. Jiao, M. Liang, H. Chen, S. Yang, H. Liu, and X. Cao, "Deep fully convolutional network-based spatial distribution prediction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 10, pp. 5585–5599, Oct. 2017.

[58] Y. Shen *et al.*, "Efficient deep learning of nonlocal features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 6029–6043, Jul. 2021.

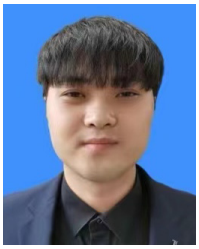
[59] J. M. Haut, M. E. Paoletti, J. Plaza, A. Plaza, and J. Li, "Visual attention-driven hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 8065–8080, Oct. 2019.

[60] L. Mou and X. X. Zhu, "Learning to pay attention on spectral domain: A spectral attention module-based convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 110–122, Sep. 2020.



Liguo Wang (Member, IEEE) received the M.S. degree in natural science and the Ph.D. degree in engineering from the Harbin Institute of Technology (HIT), Harbin, China, in 2002 and 2006, respectively.

He was a Post-Doctoral Researcher with the College of Information and Communications Engineering, Harbin Engineering University, Harbin, from 2006 to 2008, where he is currently a Professor. He has published four books, 35 patents, and more than 230 papers in journals and conference proceedings. His research interests include remote sensing image processing and machine learning.



Heng Wang received the B.S. degree in computer and communication engineering from the Zhengzhou University of Light Industry, Zhengzhou, China, in 2015, and the M.S. degree in information and communication engineering from Harbin Engineering University, Harbin, China, in 2019, where he is currently pursuing the Ph.D. degree with the College of Information and Communication Engineering.

His research interests include computer vision, deep learning, remote sensing, and deep generative models.



Lifeng Wang received the B.S. and M.S. degrees from Northeast Agricultural University, Harbin, China, in 2014 and 2017, respectively. She is currently pursuing the Ph.D. degree with the College of Information and Communication Engineering, Harbin Engineering University, Harbin.

Her research interests include hyperspectral image change detection and deep learning.



Xiaoyi Wang received the M.S. degree from Heilongjiang University, Harbin, China, in 2018. She is currently pursuing the Ph.D. degree with the College of Information and Communication Engineering, Harbin Engineering University, Harbin.

Her research interests include remote-sensing image fusion and hyperspectral target detection.



Yao Shi received the B.S. and M.S. degrees from the Department of Information and Communications Engineering, Harbin Engineering University, Harbin, China, in 2011 and 2014, respectively, where she is currently pursuing the Ph.D. degree.

Her current research interests include machine learning and remote sensing imagery processing.



Ying Cui received the Ph.D. degree in signal and information processing from Harbin Engineering University, Harbin, China, in 2006, with a focus on intelligent processing of remote sensing images, medical images, underwater images, voice, video, and other information.

Her research interests include artificial intelligence, deep learning, image classification and feature extraction, image enhancement and target detection, and network self-organization.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:27:27 UTC from IEEE Xplore. Restrictions apply.